Nobody Nose
An Evaluation of Nanotube-Based Olfactory Sensory for Neonatal Applications

Master's Thesis





Leonard Haddad

Matriculation Nr.: 3183338
leonard.haddad@uni-bremen.de

1. Reviewer: Prof. Dr.-Ing. Horst K. Hahn
2. Reviewer: Prof. Dr. med. Axel Heep

October 29, 2025

Disclaimer

I hereby declare that I have solely composed this thesis on my own and have not used any external sources except for the ones denoted as such. All content taken literally or analogously from external works has properly been indicated as such and is included under the Bibliography section.
Artificial Intelligence tools have been used solely to assist with grammer and spelling without altering context. The text, in its entirety, is human made.

Leonard Haddad

October, 2025
Bremen

Abstract

Olfactory cues play a crucial role in early human development due to their involvement in essential processes like parent-child bonding and breastfeeding. Neonates requiring clinical care are exposed to an olfactory environment that differs substantially from that of healthy children: close contact with the mother is interrupted, and natural olfactory signals are replaced or masked by disinfectants and other chemical substances. These exposures have been associated with stress responses that may negatively affect neurological development. Nevertheless, monitoring of the olfactory environment within incubators is not currently part of routine clinical care.

This study evaluates the capabilities of a novel gas detection device, the "SmellInspector", for detecting disinfectants in a simulated incubator environment and explores the feasibility of implementing autonomous, continuous olfactory sensing in neonatal care. Using semi-conducting single-walled carbon nanotube arrays, the device was tested on two commonly used disinfectants under controlled conditions. Real-time data was collected and analyzed using pattern-recognition based machine learning algorithms.

The findings suggest that although the sensor detected chemical signals, it lacked sufficient reliability to accurately distinguish disinfectants from air. Measurement drift, limited sensitivity, and environmental factors were identified as key challenges. Future work will focus on sensor optimization and alternative sensing technologies, such as metal-oxide based sensors, to enable robust odor monitoring in neonatal incubators.

Acknowledgements

I would like to express my sincere gratitude to Dr. rer. nat. Cornelia Hinz for suggesting this research topic. Exploring the technical field of odor detection has been an engaging and insightful journey.
I am also grateful to Dr. Matthias Lange from the NICU at the Children's Hospital in Oldenburg for sharing his practical experience, which greatly aided the research.
Special thanks are extended to Dr. André Homeyer and Dr.-Ing. Tom Koller for their continuous support and guidance throughout the past year.
Finally, I would like to thank Prof. Dr. med. Axel Heep for providing the opportunity to conduct research on this fascinating topic, and to both Prof. Heep and Prof. Dr.-Ing. Horst K. Hahn for reviewing this work.

Contents

1 Introduction **1**
- 1.1 Goals . . . 1
- 1.2 Structure . . . 2

2 Background **3**
- 2.1 Olfactory Cues in Neonates . . . 3
- 2.2 Challenges for in-situ Odor Measurement . . . 4
  - 2.2.1 Smell.iX16 . . . 5
  - 2.2.2 SmellInspector and SmellAnnotator . . . 5
  - 2.2.3 Implications for ongoing research . . . 6
- 2.3 Machine Learning . . . 7
  - 2.3.1 Motiviation . . . 8
  - 2.3.2 k-Nearest Neighbours (KNN) . . . 8
  - 2.3.3 Random Forest (RF) . . . 9

3 Experimental Setup **11**
- 3.1 Initial Tests and Challenges . . . 11
  - 3.1.1 Sensor Holder . . . 12
  - 3.1.2 Alcohol Tests . . . 13
  - 3.1.3 Initial Observations and Defective Sensors . . . 14
- 3.2 Establishing a standardised test-procedure . . . 15
  - 3.2.1 Initial standardised test procedure . . . 15
  - 3.2.2 Standardised testing . . . 15
  - 3.2.3 Sample Sizes . . . 17

4 Software Implementation **19**
- 4.1 Architecture . . . 19
- 4.2 The “Backend” Component . . . 20
  - 4.2.1 UART (Serial) Communication . . . 21
- 4.3 Frontend . . . 23
  - 4.3.1 Dashboard . . . 23
  - 4.3.2 Device Overview . . . 23
  - 4.3.3 Device Management . . . 24
  - 4.3.4 Adding a new device . . . 25
  - 4.3.5 Substance Management . . . 26
  - 4.3.6 Test Management . . . 28
- 4.4 Test details page . . . 29
- 4.5 Machine Learning . . . 31

4.5.1 ML-Adapters . . . 31
4.5.2 Model Training and Continuous Re-Training . . . 32
4.5.3 Data Manipulation . . . 33
4.5.4 Real-Time Inference . . . 34

5 Results and Evaluation 35
5.1 Single-Sample Classification . . . 35
5.2 Data Preparation . . . 35
5.2.1 Further Adjustments . . . 37
5.2.2 Data Re-Labelling . . . 37
5.3 Substance Classification Results . . . 39
5.3.1 Unbalanced, No Re-Labelling . . . 39
5.3.2 Undersampled, No Re-Labelling . . . 41
5.3.3 Oversampled, No Re-Labelling . . . 42
5.3.4 Unbalanced, Re-Labelled . . . 43
5.3.5 Oversampled, Re-Labelled . . . 44
5.3.6 Undersampled, Re-Labelled . . . 45
5.4 Substance Quantity Classification Results . . . 46
5.5 Humidity as a Feature? . . . 47
5.5.1 Humidity: Pitfalls . . . 49
5.5.2 SmellAnnotator: Data Classification . . . 49
5.5.3 Classification Based On Humidity Alone . . . 51
5.6 Discussion . . . 52
5.6.1 Varying Sensor Reactivity . . . 52
5.6.2 Sensor Aging . . . 53
5.6.3 Larger Test Medium . . . 53
5.6.4 Conclusion . . . 53

6 Conclusion and Future Plans 54

7 Appendix 56
7.1 Installation and Setup . . . 56
7.1.1 Required Software and Drivers . . . 57
7.1.2 UART Driver Installation . . . 57
7.1.3 Web-Configuration . . . 57
7.1.4 Machine Learning Configurations . . . 57
7.1.5 First Start . . . 58
7.2 Classification Results Overview . . . 59

Bibliography 63

List of Figures

2.1 Picture of an Incubator 4
2.2 Gas-sensing devices: chip and handheld detector. 5
2.3 Visualisation of the KNN algorithm for $k = 3$ 9
2.4 Visualisation of the RF algorithm for a forest of N Decision Trees 10

3.1 Initial tests with sensor suspended above coffee samples 11
3.2 Sensor holding mechanism sketch 12
3.3 Sensor holder with smellinspector 13
3.4 Data acquired from the new sensor (left) versus from the old sensor (right) 14
3.5 Modified experiment setup 16

4.1 The SmellInspector-Companion Software's Architecture 20
4.2 The Backend's Architecture 21
4.3 Table Showcasing 16 Channels for each of the four iX16 Sensor 22
4.4 The Dashboard page 23
4.5 Device overview page 24
4.6 Device management page 24
4.7 Device management page for a disconnected device 25
4.8 Add new device page 26
4.9 Substance management screen 27
4.10 New substance page 27
4.11 Test overview page 28
4.12 New test page 29
4.13 Test details page 30
4.14 The machine learning component's architecture 31
4.15 Initial ML-Model Training Sequence Diagram 32
4.16 ML-Model Re-Training Sequence Diagram 33
4.17 Live inference results as seen in Frontend 34

5.1 Sample Grouping of Test Data 36
5.2 Train / Test Split of Shuffled Sample Groups 36
5.3 Re-Labeling Algorithm 38
5.4 Dataset after re-labeling 38
5.5 KNN and Random Forest Confusion Matrices 40
5.6 KNN and Random Forest Confusion Matrices 41
5.7 KNN and Random Forest Confusion Matrices 42
5.8 KNN and Random Forest Confusion Matrices 44
5.9 KNN and Random Forest Confusion Matrices 45
5.10 KNN and Random Forest Confusion Matrices 46

5.11 KNN and Random Forest Confusion Matrices 47
5.12 KNN and Random Forest Confusion Matrices 48
5.13 KNN and Random Forest Confusion Matrices 50
5.14 KNN and Random Forest Confusion Matrices 52
5.15 Sensor Reactivity to "Domol" After 3 Hours (Left) vs. After 7 Hours (Right) 52

7.1 SmellInspector-Companion Software Abstract Architecture 56

List of Tables

3.1 Tested Substances' Chemical Compositions 13
3.2 Initial standardised test procedure 15
3.3 Standardised test procedure 17

5.1 Train / Test Sample Distributions 39
5.2 Statistics for raw, unbalanced data 40
5.3 Train / Test Sample Distributions 41
5.4 Statistics for undersampled, balanced data 41
5.5 Train / Test Sample Distributions 42
5.6 Statistics for oversampled, balanced data 42
5.7 Train / Test Sample Distributions 43
5.8 Statistics for re-labelled, unbalanced data 44
5.9 Train / Test Sample Distributions 44
5.10 Statistics for oversampled, re-labelled data 44
5.11 Train / Test Sample Distributions 45
5.12 Statistics for undersampled, re-labelled data 45
5.13 Train / Test Sample Distributions 46
5.14 Substance quantity classification statistics 47
5.15 Train / Test Sample Distributions 48
5.16 Classification Statistics when using Humidity as a Feature 48
5.17 Train / Test Sample Distributions 50
5.18 Classification Statistics when Averaging across each smell.iX16 Sensor . . 50
5.19 Sensor Resistance Across All Four smell.iX16 Sensors in Contrast to Computed Average Resistance 51
5.20 Classification Statistics when Averaging across each smell.iX16 Sensor **without** using Humidity 51
5.21 Classification Statistics when using only Humidity 51

7.1 Classification Results Overview 59

List of Acronyms

VOC	Volatine Organic Compound	1
NICU	Neoneonatal Intensive Care Unit	1
ML	Machine Learning	2
PCA	Principal Component Analysis	7
LDA	Linear Discriminant Analysis	7
KNN	k-Nearest Neighbours	8
RF	Random Forest	9

Chapter 1

Introduction

Olfactory cues play a crucial role in early human development due to their involvement in essential processes like parent-child bonding and breastfeeding. Neonates requiring clinical care are exposed to an olfactory environment that differs substantially from that of healthy children: close contact with the mother is interrupted, and natural olfactory signals are replaced or masked by disinfectants and other chemical substances. These exposures have been associated with stress responses that may negatively affect neurological development. Nevertheless, monitoring of the olfactory environment within incubators is not currently part of routine clinical care, especially due to high sensor costs, large space requirements of novel gas spectroscopes and due to lacking research regarding relevant Volatine Organic Compound (VOC) relevance on neonatal development.
SmartNanotubes GmbH, a German-based startup, in collaboration with the Technological University Dresden (TU Dresden) and partners from Finland and Israel, have developed the "iX16" gas detection chip. Marketed as the "world's first semiconducting carbon nanotube gas detection chip" [1], the sensor promises gas detection at the parts per billion (ppb) scale, offering a sensitivity comparable to that of state-of-the-art gas spectroscopes. In addition, it claims to offer automated gas classification through the accompanying "SmellAnnotator" software [2, 3], which employs machine learning models that perform gas classification based solely on pattern-recognition.
If these performance claims hold true, the technology has the potential to facilitate the use of compact and cost-effective gas sensors in neonatal clinical settings and to open up a wide range of applications from cancer detection to rapid medical diagnostics in remote or resource-limited environments.

1.1 Goals

The overarching goal of this research is to evaluate the feasibility of integrating lightweight olfactory monitoring technology into the neonatal clinical routine at the Children's Hospital in Oldenburg, and the Neoneonatal Intensive Care Unit (NICU) routine in general. To achieve this, this following objectives are persued:

1. **Sensor Capability Assessment:** Investigate the detection limits and reliability of the SmellInspector in controlled settings to assess the feasibility of clinical applications based on carbon nanotube sensors.

2. **Substance Classification:** Evaluate the ability of the SmellInspector to accurately classify common disinfectant liquids used in the clinical environment, using

pattern-recognition based Machine Learning (ML) models.

3. **Real-Time Monitoring Feasibility:** Assess whether the ML models can detect and classify target substances in real time under realistic neonatal care conditions.

4. **Clinical Integration Potential:** Determine the practical considerations for the permanent deployment of the technology in neonatal wards, including installation requirements, maintenance, and usability by medical personnel.

These objectives will help determine the technological and clinical readiness of the SmellInspector for integration into routine neonatal care.

1.2 Structure

Chapter 2 establishes the theoretical foundation of this work by examining olfactory cues and their influence on neonatal development. It introduces SmartNanotubes GmbH and their nanotube-based sensing technologies, highlighting their relevance for precise odor detection. Additionally, the chapter provides an overview of machine learning (ML) principles, emphasizing pattern recognition models that underpin the analytical approach employed throughout this research.
Building on this foundation, Chapter 3 presents the experimental setup used for data acquisition with the SmellInspector device. It details the tested substances, the configuration of the sensor system, and the procedures followed to ensure reproducibility and accuracy in measurements.
Chapter 4 focuses on the software developed to facilitate data acquisition, real-time monitoring, and analysis. It also describes the ML models implemented for substance classification and outlines their integration within the data processing workflow.
Following the software description, Chapter 5 offers a comprehensive evaluation of the classification algorithms, analyzing their performance across different substances, concentrations, and experimental conditions, and highlighting strengths and limitations of each model.
Finally, Chapter 6 synthesizes the results, discussing their implications for potential clinical applications, evaluating the feasibility of deploying nanosensor-based odor detection in neonatal care, and outlining directions for future research in this domain.

Chapter 2

Background

This chapter provides an overview of olfactory cues in neonates and their role in early sensory and cognitive development. It explores how exposure to specific olfactory stimuli may influence physiological regulation, emotional bonding, and learning processes during the neonatal period. Furthermore, it introduces the nanotube-based gas-sensing technology employed throughout this research and potential advantages for detecting Volatile Organic Compounds (VOCs) in neonatal environments. Finally, an introduction to machine learning (ML) is provided, outlining its relevance to odor pattern recognition and classification.

2.1 Olfactory Cues in Neonates

The sense of smell, though often overlooked in early developmental research, plays a complex and essential role in shaping neonatal behavior, attachment, and physiological regulation. This raises important questions regarding how olfactory cues influence the earliest stages of human development.

While full-term neonates experience a rich and familiar olfactory environment in utero, preterm infants are born into settings with markedly different sensory conditions. Within the womb, maternal odors derived from amniotic fluid, maternal diet, and metabolic processes, provide consistent olfactory stimulation [4]. In contrast, the NICU exposes preterm infants to an environment dominated by sterile air, artificial nutrition, and antiseptic or chemical odors, largely devoid of maternal olfactory cues.

Preterm infants frequently experience complications such as feeding difficulties, apnea, and long-term cognitive, emotional, or social challenges, including disruptions in early emotional bonding. These issues highlight the importance of understanding how sensory deprivation or alteration of olfactory input may contribute to typical developmental trajectories. Although the human sense of smell was once considered relatively underdeveloped, emerging research suggests that olfaction exerts a more influential role in early life than previously assumed [5]. For example, exposure to maternal body odors has been shown to reduce infants' neural responses to fearful facial expressions [6], indicating a potential stress-modulating effect. Moreover, although the olfactory system is not fully mature at birth, evidence from MRI studies demonstrates that fetal discrimination of odorant molecules in amniotic fluid is possible after approximately 28 to 30 weeks of gestation [7]. Additionally, neonates can recognize their mothers through olfactory cues learned prenatally [8], emphasizing the continuity of olfactory learning and its relevance for early attachment and regulation.

Furthermore, exposure of neonates to harsh artificial odors has been associated with stress responses that may adversely affect neurological development [9, 4]. Although research remains inconclusive, especially due to the methodological challenges of empirically testing olfactory hypotheses in-situ, current research suggests that olfactory stimulation might contribute to the neurological development of preterm infants. Despite this, systematic monitoring and management of the olfactory environment within incubators has not yet been integrated into the standard clinical practice.

2.2 Challenges for in-situ Odor Measurement

To empirically evaluate the effects of olfactory factors and to optimize environmental conditions for neonates, gas sensing technologies must be integrated directly with incubators. To date, Volatile Organic Compound (VOC) analysis has primarily been performed on fecal samples or from the headspace above diapers, successfully demonstrating the potential for detecting conditions such as late-onset sepsis [10, 11], necrotizing enterocolitis [12], and bronchopulmonary dysplasia [12]. However, studies performing direct VOC analysis within incubators remain rare, with only one study investigating VOC patterns to determine incubator occupancy [13].



Figure 2.1: Picture of an Incubator

The placement of olfactory sensors presents several challenges, including bulkiness, cost, and invasiveness. As illustrated in Figure 2.1 (Taken from [14]), sensors can either be connected to the incubator's air circulation system, which may reduce the amount of breathable air available to the neonate, to the exhaust fan on the bottom of the incubator, requiring a small scale sensor, or mounted directly inside the incubator. The latter approach requires sensors that are minimally invasive and compact while maintaining sufficient accuracy for VOC detection to ensure reliable measurements.
Furthermore, the integration of olfactory sensors into clinical practice is complicated by additional factors, including the largely unknown effects of diverse Volatile Organic Compounds (VOCs) on neonatal development, as well as the technical challenges associated with reliable data acquisition, evaluation, and sensor technology selection. Although previous studies have examined the influence of certain odors in addressing the aforementioned challenges faced by neonates [4], these investigations generally assessed the

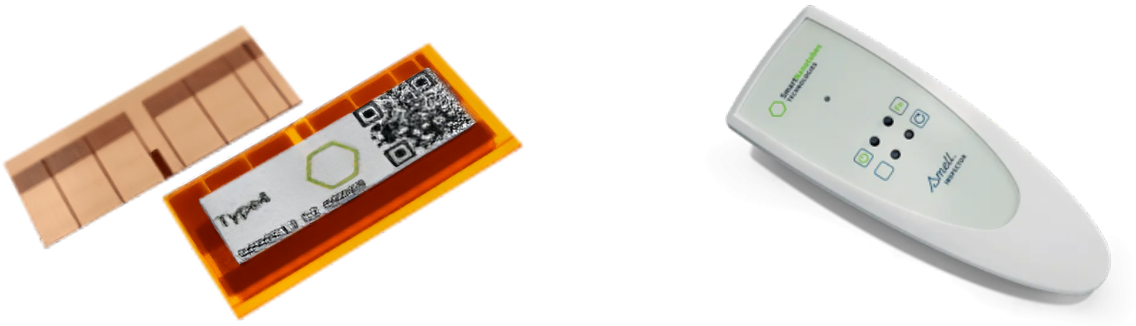
effects of odor exposure on a broader, perceptual level rather than conducting detailed chemical analyses of the specific VOCs involved. As a result, the precise biochemical and physiological pathways through which odorants impact neonatal health remain poorly understood.

2.2.1 Smell.iX16

SmartNanotubes Technologies GmbH is a german startup based in Freital, Germany, that has, in collaboration with the Technological University Dresden (TU Dresden) and partners from Finland and Israel, developed the "world's first semiconducting carbon nanotube chip" [1].
The chip, dubbed the "smell-iX16" consists of an array of 16 AuNP-functionalized sc-SWCNT sensors (gold-nanoparticle-coated, semi-conducting, Single-Walled Carbon NanoTubes), that perform gas detection. Although all 16 sensors are fabricated to identical specifications, minor variations in sensor resistance enable the detection of a diverse range of gases and Volatile Organic Compounds (VOCs). These variations are believed to arise from the dispersion of N-methyl-2-pyrrolidone across the array during fabrication, as well as from slight imperfections in the microelectrodes of individual sensors [15].

2.2.2 SmellInspector and SmellAnnotator

As a gas sensing product, SmartNanotubes manufactures and sells the "SmellInspector" device, a portable, handheld gas sensing device that houses four smell.iX16 gas detection sensor arrays. By itself, the device is only able to record and output resistance data (in ohms) via a USB UART interface to a connected PC, or via Bluetooth to a connected mobile device. The connected device then performs gas detection and classification.



(a) smell.iX16 AuNP-functionalized sc-SWCNT Gas sensing chip

(b) SmellInspector handheld gas sensing device

Figure 2.2: Gas-sensing devices: chip and handheld detector.

To work with sensor data, SmartNanotubes developed the "SmellAnnotator" [2, 3] software that can plot and analyse the data, and perform gas classification using integrated Machine Learning (ML) models. As a proof of concept, the software ships with a coffee detection model, that can differentiate between ambient air, the presence of a foreign gas (unknown gas other than ambient air) and between the smell of coffee. Further models need to be trained manually, a process that SmartNanotubes offers its industrial clients as a service. Alternatively, individual users can annotate acuired data in the SmellAnnotator

software and train machine learning models, with the ML models remaining black-boxes with unknown parameters. Details regarding the functionality of the classification models are not provided, citing only "pattern-based" recognition technologies.
According to specifications, the SmellInspector can achieve gas detection and classification performance that is comparable to, or even exceeding, the capabilities of current industrial systems at a fraction of the cost [16]. The device is designed for long-term industrial use, requiring minimal calibration and promising long-term functionality. Furthermore, the power draw of only $0.28W$ allows for prolonged operation while using a pair of over-the-counter AAA batteries as a power source for portable operations, though data acquisition still requires an accompanying mobile device or laptop. Further information regarding the sensor is provided in Chapter 3.
When this research was started, little was known about the detection accuracy of the smell.iX16 chips and thus various tests were carried out as outlined in Chapter 3. As of now, SmartNanotubes has released a list of tested substances that are detectable by the chips. The list can be found in the SmellInspector's user manual [17].

2.2.3 Implications for ongoing research

Within the Perinatal Research Group at the Carl von Ossietzky University of Oldenburg [18], ongoing investigations focus on the influence of olfactory cues on the cognitive and physiological development of preterm infants [4]. Manipulation of olfactory cues has already been proven to combat feeding problems and apnea and has thus been integrated into the clinical routine at the NICU in the Children's Hospital in Oldenburg.
Among the environmental factors studied, exposure to artificial odors has emerged as a key concern. Recent research indicates that artificial odors can elicit stress responses in infants comparable to those triggered by other environmental stressors, potentially influencing long-term cognitive and emotional outcomes. However, current research efforts are limited by the use of bulky and expensive sensory equipment, restricted data availability, and an incomplete understanding of the specific effects of Volatile Organic Compounds (VOC) on physiological development. Consequently, such sensors are not yet incorporated into routine clinical care and often remain confined to controlled laboratory settings.
In this context, a compact, non-intrusive, and lightweight sensing device such as the SmellInspector represents a promising tool for empirically evaluating the effects of VOC and artificial odors on neonatal development in long-term clinical settings. If demonstrated to be reliable in odor classification, the device could be integrated directly into incubators to continuously monitor air quality while simultaneously collecting valuable research data. The resulting data could not only advance the refinement of classification algorithms but also facilitate the development of preventive measures, such as automated alarm systems that alert clinical staff when concentrations of disinfectant agents or other harmful substances exceed safe thresholds, thereby reducing unnecessary exposure of infants, or issue notifications when infants require a diaper change, thereby reducing unnecessary physical contact and minimizing potential stress for neonates. If proven effective, such systems could also serve as early diagnostic tools for detecting common neonatal conditions, including infections or gastrointestinal disorders, through characteristic changes in VOC profiles.
Ultimately, the integration of such technology into routine neonatal care holds the potential to bridge the gap between experimental research and clinical application, resulting in

a deeper scientific understanding of the human olfactory system and its effects in early development, and improve safety and well-being for preterm infants within the NICU. To perform odor and gas classification, statistical models are used, dubbed with the term "Machine Learning (ML)", as explained in the following section.

2.3 Machine Learning

The term "machine learning" (ML) refers to the ability of a computer to learn without being explicitly programmed for a particular task, as coined by Arthur Samuel in 1959 [19]. It denotes the field in which computers are fed large amounts of data with the goal of detecting and learning from patterns within the data for a specific task. In the field of ML, we differentiate between:

1. **Supervised Learning:** The ML model is fed inputs (data) with known outputs (labels), with the goal of learning a robust mapping between the input data and the output labels that generalizes to unseen data.

2. **Semi-Supervised Learning:** The dataset fed to the model consists of both labeled and unlabeled data. The model learns from the labeled data and then attempts to infer information about the unlabeled data.

3. **Unsupervised Learning:** In unsupervised learning, the model is provided only with inputs, with the goal of identifying correlations and relationships within the input data, which can later be used to solve clustering, association, or dimensionality reduction problems.

4. **Reinforcement Learning:** Unlike other paradigms, in reinforcement learning the model continuously learns from past experiences to determine the optimal actions or behaviours to perform for a specific task.

When dealing with large amounts of data, such as those gathered by the SmellInspector, either the supervised or semi-supervised approaches are generally used. While supervised learning ensures that the model captures the underlying structure of the entire dataset, this approach often results in a large computational overhead due to the extensive labeling effort required for large datasets. Therefore, semi-supervised learning is often preferred. The process of training a ML model involves four stages:

1. **Data Gathering:** Collecting data that will be fed to the model, either labeled or unlabeled.

2. **Data Modeling:** Before training a ML model, the gathered data is modeled and examined for underlying correlations and structures. This process involves the use of pattern recognition techniques such as Principal Component Analysis (PCA) [20] and Linear Discriminant Analysis (LDA) [21].

3. **Model Training:** The input/output data pairs are fed to the ML model. The model analyzes the data and attempts to learn correlations between the input data and the corresponding outputs. The choice of model depends on the nature of the problem.

4. **Evaluation:** The model's performance is assessed using common metrics such as "accuracy", "precision", "recall", and "F-score":

   - **Accuracy:** Measures the proportion of correctly predicted samples among all samples. It reflects the overall correctness of the model.
   - **Precision:** Measures the proportion of correctly predicted positive samples among all samples predicted as positive. It indicates how reliable the model's positive predictions are.
   - **Recall (Sensitivity):** Measures the proportion of correctly predicted positive samples among all actual positive samples. It evaluates the model's ability to identify positive cases.
   - **F-score (F1-score):** Represents the harmonic mean of precision and recall, providing a single measure that balances both metrics.

The training and evaluation process can be repeated as often as necessary to achieve a desired output mapping.

2.3.1 Motiviation

When dealing with high-dimensional data, ML is often used alongside dimensionality-reduction tools to train robust models that generalize to large, unseen data. Since the SmellAnnotator software already uses ML models [1] for identifying gas measurements in real-time, and includes built-in tools for data analysis (PCA and LDA), this research follows a similar approach.

2.3.2 k-Nearest Neighbours (KNN)

One of the simplest and most intuitive supervised learning algorithms is the k-Nearest Neighbours (KNN) method. Originally introduced by Evelyn Fix and Joseph L. Hodges Jr. in 1951 [22], the algorithm classifies an unseen sample based on its proximity to previously labelled samples within a multidimensional feature space.
The underlying assumption of KNN is that samples with similar characteristics (those located close together in the feature space) are likely to belong to the same class.
To determine the class label of a new sample, the algorithm identifies the k closest labelled samples (the k nearest neighbours) and assigns the majority class among them to the new sample. The parameter k is typically chosen as a small odd integer, often between 3 and 7. Figure 2.3 (Taken from [23]) visualizes the KNN decision process for a two-class problem with $k = 3$.
To compute the k nearest neighbours, the Euclidean distance between the new sample and all other samples is typically calculated across n dimensions. This process must be repeated for each prediction, resulting in a computational cost that increases linearly with both the number of samples and the dimensionality of the data. While the algorithm itself is conceptually simple, it can become computationally expensive for large datasets.

[1] The SmellAnnotator software comes with a built-in coffee classification model. Which underlying ML model is used is not mentioned

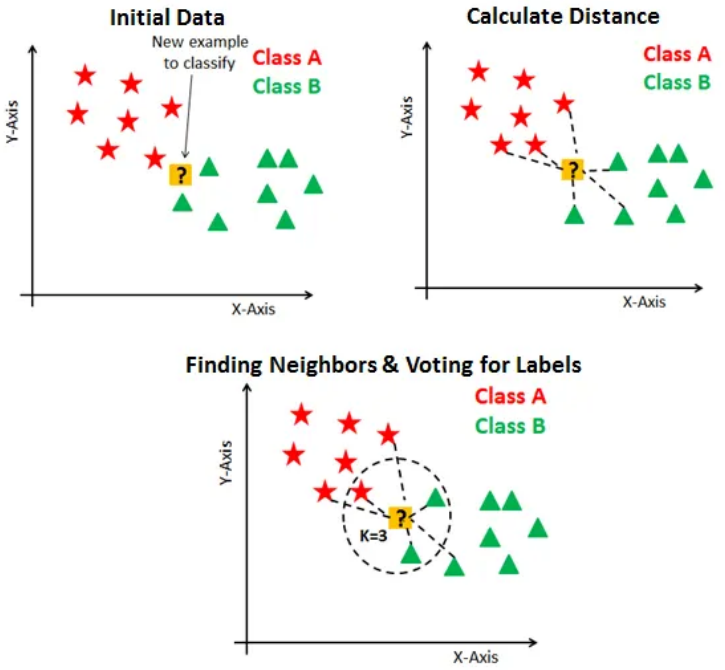


Figure 2.3: Visualisation of the KNN algorithm for $k = 3$

Because KNN relies solely on distance metrics, it performs well on well-structured, low-dimensional data. However, its performance tends to degrade on complex or high-dimensional datasets, especially when features are correlated or the data distribution changes over time, as is often the case with continuous sensor measurements subject to drift. For this reason, the KNN model was employed primarily as a baseline classifier to compare performance against more robust models capable of capturing deeper feature relationships.

2.3.3 Random Forest (RF)

The Random Forest (RF) algorithm is a powerful ensemble learning method introduced by Leo Breiman and Adele Cutler in 2001 [24]. It builds upon earlier concepts such as bagging and random subspace methods to create a robust model for both classification and regression tasks.
In 1993, Salzberg and Heath proposed the concept of random decision forests, utilizing a randomized decision tree algorithm to generate multiple distinct trees and combining their outputs using majority voting [25]. This idea was further refined by Tin Kam Ho in 1995, who introduced the method of random subspace selection, where each tree in the forest is trained on a random subset of features, enhancing diversity among the trees [26]. Building upon these concepts, Breiman and Cutler formalized the RF algorithm by integrating "bagging", where variance is reduced by training models on subsets of the data, with the random feature selection approach. This combination resulted in a model that is less prone to overfitting and capable of handling large datasets with high dimensionality. When used for a classification problem, RF constructs a multitude of decision trees during training and outputs the class that is the mode of the classes of the individual trees.
To achieve this, the algorithm performs the following operations:

1. **Bagging:** Generate multiple "bootstrap" samples from the original dataset and train a decision tree on each sample.

2. **Random Feature Selection:** At each node, select a random subset of features and determine the best split based on this subset, promoting diversity among the trees.

3. **Voting/Averaging:** Each tree casts a vote for the predicted class, and the class with the majority vote is chosen.

Figure 2.4 (Taken from [27]) illustrates the decision and voting procedure.

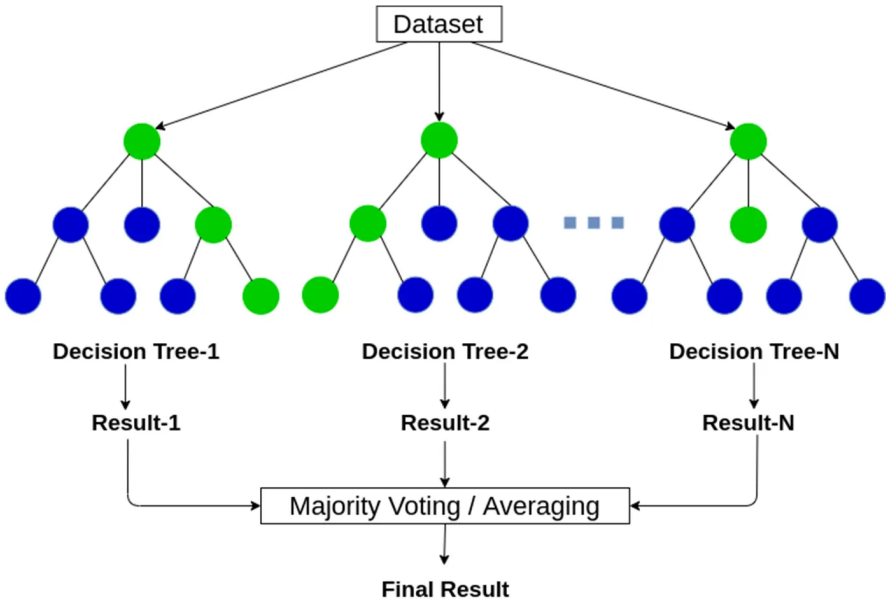


Figure 2.4: Visualisation of the RF algorithm for a forest of N Decision Trees

This ensemble approach leads to a model that is more accurate and stable than individual decision trees and a far more robust approach as compared to KNN.
Though RF classifiers often result in highly accurate and robust models, they are often hard to interpret. Furthermore, the ensemble approach with its many decision trees often results in high computational complexity, resulting in longer training times. Despite these limitations, due to their high versatility and effectiveness across a wide range of applications, they were chosen as the main classification model.
Importantly, neither the sensing technology nor the ML algorithms alone are solely responsible for accurate odor and VOC classification. Rather, successful classification depends on the interplay of several factors, namely appropriate experimental design, precise data acquisition and preprocessing, and careful selection and training of suitable ML models. Together, these components determine the overall reliability and robustness of the odor classification model.
Given the structured and relatively low-dimensional nature of the SmellInspector data, a KNN classifier and a RF classifier were chosen. This selection enables a comparison between a simple, instance-based method that relies on proximity in feature space (KNN) and a more sophisticated, ensemble-based model capable of capturing non-linear relationships and complex feature interactions (RF). Together, these classifiers provide a balanced assessment of model complexity versus interpretability. The KNN approach serves to validate the premise that odor classification can be achieved through direct pattern recognition, whereas the RF model facilitates a deeper analysis of underlying correlations and variable importance within the dataset.

Chapter 3

Experimental Setup

To evaluate the SmellInspector for in-situ operation, a series of tests were conducted to assess its detection capabilities under controlled conditions. Based on these preliminary experiments, a standardized experimental procedure was developed to enable consistent and replicable large-scale data collection.

3.1 Initial Tests and Challenges

Note: As mentioned previously, at the point of writing, litte public information was available regarding sensor sensitivity. Thus, experiments were carried out to determine what substances would fit the accompanying research.
Due to the limited availability of public data on the sensitivity and selectivity of the smell.iX16 sensors to different gases, a series of preliminary tests were conducted under both laboratory and home conditions to obtain a general understanding of the SmellInspector's responses to common substances.



Figure 3.1: Initial tests with sensor suspended above coffee samples

As the SmellInspector is supplied with a pre-trained coffee detection model within the SmellAnnotator software, initial experiments focused on evaluating its performance using various forms of coffee. These included whole coffee beans, instant espresso powder, and

brewed liquid coffee, each tested at different ambient temperatures and concentrations to observe variations in sensor response.
To minimize contamination from ambient odors, the tests were performed inside an atmospherically controlled chamber equipped with adjustable airflow valves. During each trial, the sensor was positioned approximately 2 centimetres above the sample surface using a micro-USB cable as a suspension mechanism (see Figure 3.1). Real-time data capture and visualization were carried out using the SmellAnnotator application.
Among the tested samples, only the liquid coffee produced a noticeable shift in the recorded sensor data. The built-in coffee classification model successfully identified the sample as coffee in one instance. However, subsequent repeated trials failed to replicate this result. In later tests, the system consistently classified the samples as either "air" or "other", the latter denoting the detection of an odor that does not correspond to any known reference category within the model.

3.1.1 Sensor Holder

Suspicions regarding the detection accuracy ranged from faulty sensor data to insufficient odor saturation in the surrounding air. Although the SmellInspector is marketed with detection sensitivities in the parts per billion (ppb) range, later tests indicated that the effective accuracy for the tested substances was considerably lower.
To investigate a potential lack of odor saturation, a custom sensor holding mechanism was designed and 3D printed (see Figure 3.2). The corresponding print files are available on Makerworld [28].

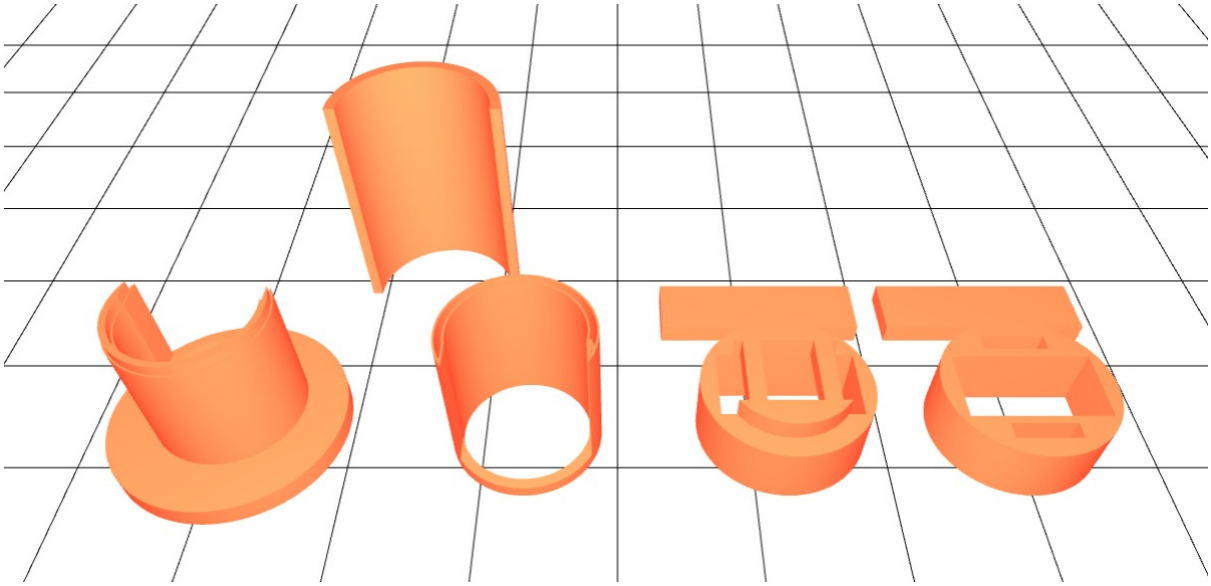


Figure 3.2: Sensor holding mechanism sketch

The mechanism consists of a circular base for the placement of 10 cm laboratory vials and a top section designed to hold the sensor, as shown in Figure 3.3. The base also includes a 3D-printed door mechanism that allows the chamber to be closed, forcing the evaporated sample to disperse upward through air holes toward the SmellInspector.
Using the enclosed sensor holder, the coffee test was repeated. Once again, the coffee model successfully detected the presence of a substance but failed to classify it as coffee.



Figure 3.3: Sensor holder with smellinspector

This outcome was unsurprising, as the model is "only provided as a proof of concept and is trained using a small dataset", according to SmartNanotubes.

3.1.2 Alcohol Tests

With the failure of the coffee model, the focus shifted to the practical use case of the SmellInspector for in-situ operations. While detecting coffee odor proved challenging, the strong odors of ethanol and propanol were expected to be more easily detectable, an assumption that was subsequently confirmed.
From this point onward, the only liquids tested were "Octeniderm" [29] and "Domol" [30]. The chemical compositions of each substance are provided in Table 3.1. Although both substances contain large quantities of propanol, the addition of ethanol and fragrances in Domol was expected to provide a sufficient distinction to differentiate between the two substances.

Substance	**Chemical Composition** / $100g$
Octeniderm farblos	Octenidine dihydrochloride: 0,1 g 1-Propanol: 30,0 g 2-Propanol: 45,0 g
Domol Hygiene-Spray	Ethanol: 22,0 g 1-Propanol: 8,0 g 2-Propanol: 21,0 g

Table 3.1: Tested Substances' Chemical Compositions

To determine the optimal test procedure for subsequent experiments, initial trials were conducted using medial vials with a diameter of 10 cm. The vials were positioned beneath the sensor, and data acquisition was performed over several hours to observe the sensor's response as the concentration of the substance in the surrounding air increased.
For data acquisition and visualization, the software described in Chapter 4 was employed. Although initial tests with coffee and alcohol used the SmellAnnotator software provided by SmartNanotubes, it quickly became apparent that its data representations did not offer sufficient insight into the sensor's raw, unmodified data.
Consequently, a custom software solution was developed and utilized for the experiments.

3.1.3 Initial Observations and Defective Sensors

Inspection of the data gathered during this stage revealed erratic fluctuations in sensor resistance, prompting multiple consultations with SmartNanotubes. Speculations regarding sensor aging and defective units were dismissed, and recommendations were made concerning "warm-up" periods prior to measurement and potential issues in data representation.
To evaluate these claims, additional tests were conducted using a "warm-up" duration of four hours. During this time, the sensor remained suspended in its cage without any vials beneath it, measuring only ambient air. Following the warm-up period, measurements were conducted as before. Simultaneously, the developed software's data representations were checked for errors, and the raw sensor data was manually inspected for validation.
When these steps failed to improve the results, filtering algorithms were applied to remove sensor readings in the Megaohm range. Although this stabilized the data for 61 of the 64 sensor channels, it also reduced the overall data dimensionality, introducing a bias with unknown implications for subsequent classification.
As a result, a new SmellInspector device was obtained from SmartNanotubes. When both devices were connected to the software and operated side by side, the presence of faulty sensors in the original device was confirmed, as illustrated in Figure 3.4.

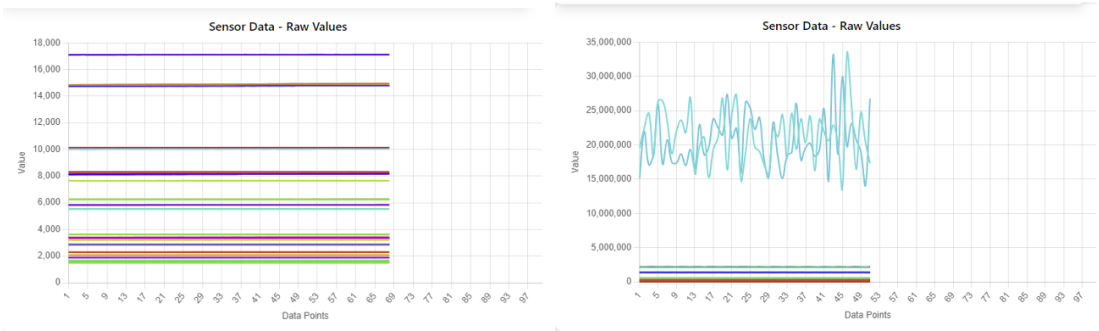


Figure 3.4: Data acquired from the new sensor (left) versus from the old sensor (right)

The cause of the sensor defects was never established. The "old" sensor was 1.5 years old at the start of the experiments. The "new" sensor was acquired in Q4/2024 and continues to provide reliable data as of October 2025. The old sensor was never re-calibrated and continues to produce faulty readings.
After repeating the experiment with a vial placed beneath the sensor, the new device exhibited consistent and repetitive response patterns whenever exposed to the disinfectant liquids. The resistance values fluctuated gradually over several hours, reflecting changes in environmental conditions, namely humidity and temperature.

3.2 Establishing a standardised test-procedure

With a functional sensor available, efforts were focused on developing a standardised and repeatable experimental test procedure.

3.2.1 Initial standardised test procedure

Following the manufacturer's recommendations regarding warm-up times and to allow for natural evaporation of the liquids, it was decided to run tests in 30-minute intervals. Each test was divided into 30-minute windows, during which the sensor was initially exposed to ambient air. A sample was then introduced, and the sensor's response was recorded over a 30-minute period while the substance evaporated. Afterward, the ambient air was recycled using the air valves of the gas chamber. This cycle was repeated for successive samples as frequently as possible, maintaining consistent timing and environmental control for each measurement. During the air-recycling period, the sensor continued to record data, which was labeled as "air".

	Test Setup (Once)		Repeated	
Duration	**<4 hours**	**After 4 hours, for 30 Minutes**	**30 Minutes**	**30 Minutes**
Function	Warm-up (No measurement)	Initial, ambient air measurement	Introduction and dissipation of test sample	Air recycling / ambient air measurement

Table 3.2: Initial standardised test procedure

To validate this procedure, a series of tests was conducted over the course of two months. Although the tests were standardised and repeatable, the sensor frequently failed to detect the presence of a sample, primarily due to the slow evaporation rate. For larger sample volumes (above 50 μl), complete dissipation could take up to an hour, rendering the procedure impractical. Attempting to classify the samples from the collected data was also not feasible, as most measurements taken during the 30-minute intervals contained no discernible signal from the sample and were effectively indistinguishable from ambient air readings.

3.2.2 Standardised testing

To establish a reliable test procedure capable of generating usable data, the experimental conditions were modified as follows:

- The previously designed sensor enclosure 3.3 was split in half. The bottom section, which housed the test vials, was removed and replaced with a 1.65 L plastic box (60 mm × 145 mm × 190 mm) designed to mimic an incubator on a smaller scale. The modified enclosure included two openings: one on the top for mounting the SmellInspector and one on the side for liquid injection, as shown in Figure 3.5.

- Instead of relying on natural evaporation, an off-the-shelf coffee heating plate [31] was used to accelerate dissipation, reducing the average evaporation time from several minutes to approximately 5 seconds.

- Air and sample measurement durations were shortened from 30 minutes to several seconds, as described below.

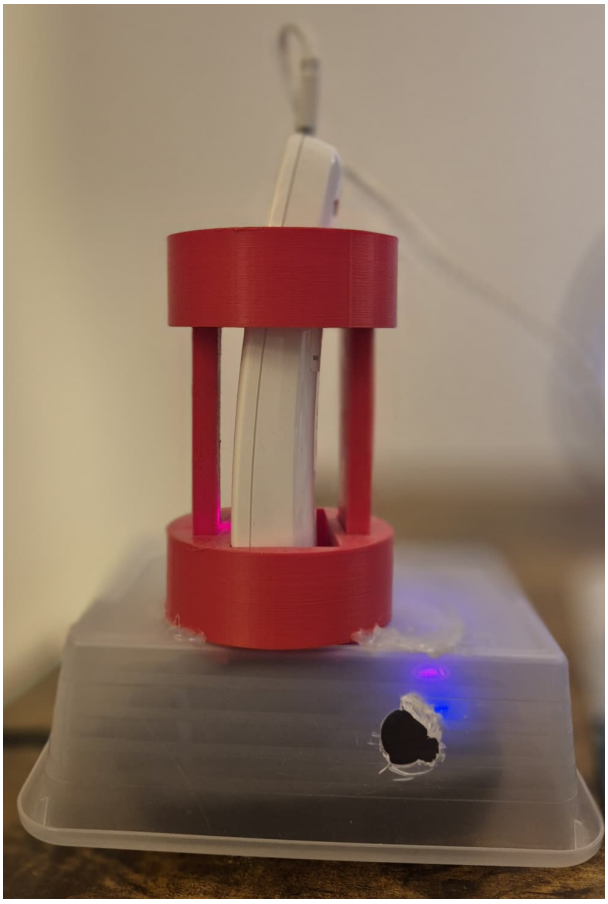


Figure 3.5: Modified experiment setup

Where initial experiments relied on fixed 30-minute measurement intervals, observation of the SmellInspector's live data feed indicated that this approach was far from optimal. With the heating plate accelerating evaporation to nearly instantaneous levels at 90°C, the sensor's reaction time decreased from several minutes to a few seconds, with readings returning to nominal pre-sample conditions within approximately 15 seconds.
To address this, a new test procedure was proposed: instead of relying on fixed time intervals, measurements would be triggered based on air humidity as measured by the SmellInspector's hygrometer. Although the temperature and humidity values reported by the sensor differ from actual ambient conditions (an external hygrometer was used for reference), these readings can still reliably indicate when the sensor detects the presence of a substance.
To determine the appropriate start time for a measurement, the SmellInspector is first operated with the previously described warm-up procedure 3.2. Once warmed up, the sensor's humidity readings remain relatively stable, fluctuating by approximately ±1% around a mean value determined by the current environmental conditions. This mean value, plus an additional allowance of up to 5% humidity, is then used as a baseline reference for determining the end of a measurement.
A measurement is then conducted as follows:

- The label of the current substance is set in the software (see Chapter 4).
- The sample is introduced. Consequently, the humidity measured by the sensor rises rapidly, reaching a peak after a few seconds. Once the sample has dissipated, the

humidity decreases again toward the baseline recorded at the start of the measurement. An average measurement typically lasts 30 to 60 seconds. See Table 3.3. *When multiple tests are conducted consecutively, the baseline ambient humidity tends to increase. Therefore, the measurement is considered complete when the humidity returns to the baseline plus up to* 5%.

- When the baseline is reached, the current substance label is reset to "air", and the experiment is repeated after a 60-second interval.

Note: The sensor continuously records data throughout the entire procedure. The recorded samples are labeled either as "air" or with the corresponding substance, depending on the current step of the experiment.

	Test Setup (Once)		**Repeated**	
Duration	**<4 hours**	**After 4 hours, for 60 seconds**	**30-60 Seconds (on average)**	**60 Seconds**
Function	Warm-up (No measurement)	Initial, ambient air measurement	Introduction and dissipation of test sample	Air recycling / ambient air measurement

Table 3.3: Standardised test procedure

Using this test procedure not only enables rapid data acquisition but also ensures that the recorded samples correspond to time intervals during which the substance is present in the surrounding air at sufficiently high concentrations. To prevent accumulation of alcohol on the walls of the plastic box, the box was inverted and allowed to rest for up to 10 minutes after every five test runs. During this period, the sensor continued recording data labeled as "air". This does not introduce a bias, as the sensor is removed from the holder during the resting period.
To verify the effectiveness of the procedure, sensor data collected before, during, and after measurements was carefully inspected. Using humidity as an indicator proved reliable and ensured accurate data acquisition. Although this procedure produced consistent results, post-processing was still required to remove remaining samples in which no substance was detected (See Section 5.3).

3.2.3 Sample Sizes

With a functional test procedure established, the next challenge was determining appropriate substance quantities for the experiments. In reference to practical use cases in the children's hospital, measurements of up to 100 μl were considered realistic in extreme cases. Typical scenarios, such as a nurse disinfecting their gloves before handling an incubated infant, would usually involve doses no larger than 20 μl, which proved insufficient for detection by the SmellInspector.
Before performing large-scale experiments using the established test procedure, the SmellInspector's detection sensitivity to various sample volumes was tested for both tested substances. Measurements were carried out with sample sizes ranging from 10 μl to 100 μl in 5 μl increments.

Based on these experiments, the following test sample sizes were selected for subsequent tests:

- 50 μl as the standard test sample size. Volumes smaller than 30 μl failed to register for Octeniderm and produced only minimal sensor responses for Domol.
- 75 μl as a slightly elevated reference sample size.
- 100 μl as an extreme-case sample size.

At volumes below 30 μl, the sensor registered barely any readings for either substance. Between 35 μl and 40 μl, small responses were observed for Domol, likely due to its strong fragrances. At 50 μl, the SmellInspector exhibited noticeable responses to both substances, and this dosage was deemed appropriate for most tests, as it also represents a realistic scenario for disinfecting medical equipment. To further assess the sensor's ability to distinguish between sample sizes, recordings were also taken for 75 μl and 100 μl, both of which produced elevated sensor responses similar to those observed at 50 μl.
With the sample sizes, experimental setup, and procedures established, a series of tests was conducted over several months. During this period, both the software and machine learning (ML) components were continuously adjusted to incorporate new insights about the sensor's behavior.
Ultimately, the established experimental procedure was designed to collect data that closely reflects conditions in clinical settings, enabling an assessment of the SmellInspector's detection capabilities and its feasibility for in-situ deployment. Achieving reliable results under these conditions is a critical step toward integrating olfactory measurement into the clinical routine. If the device demonstrates sufficient performance, subsequent research goals focusing on real-time odor monitoring and broader clinical integration can be persued, as outlines previously.
The following chapter describes the software developed and employed for these experiments, along with the machine learning (ML) models used for data capture and annotation.

Chapter 4

Software Implementation

4.1 Architecture

The software described in this section can be found on GitHub [32].
Due to the limitations of the SmellAnnotator software, the "SmellInspector Companion Software" was developed to enable data acquisition and real-time monitoring, support simultaneous use of multiple sensors from the same host device, and allow real-time inference of substances and gases using custom machine learning (ML) models.
To ensure versatility and facilitate customization for various research and clinical applications, the software was designed as three separate microservices, illustrated in Figure 4.1:

1. The web-based user interface (the "Frontend" component), which allows users to interact with connected sensors, preview current olfactory measurements, and view inference results.

2. The Python-based "Backend" component, which acts as a data proxy between the user interface, connected sensors, and the machine learning software. It handles data acquisition, storage, and returns inference data.

3. The Python-based "Machine Learning (ML)" component, which utilizes data acquired during previous tests to train various ML models and enables real-time classification of measured substances for new data.

Furthermore, both the Frontend and ML components were designed as Docker containers, enabling one-click deployment and effortless replication to support large-scale adaptability and ensure reproducibility. The "Backend" component, in contrast, is intended to run directly on the host operating system of the machine to which the SmellInspector devices are connected, due to limitations in proxying data from connected UART devices through Docker.
In contrast to the SmellAnnotator software by SmartNanotubes, the developed software does not require an account or an active internet connection for substance classification and supports data acquisition from multiple SmellInspector devices simultaneously. Recorded data is propagated to web clients in real time using WebSockets (Socket.IO), ensuring unfiltered, continuous data streams. The machine learning (ML) component provides a variety of hot-swappable, customizeable ML adapters, allowing additional adapters to be integrated easily. For the experiments described here, a Random Forest

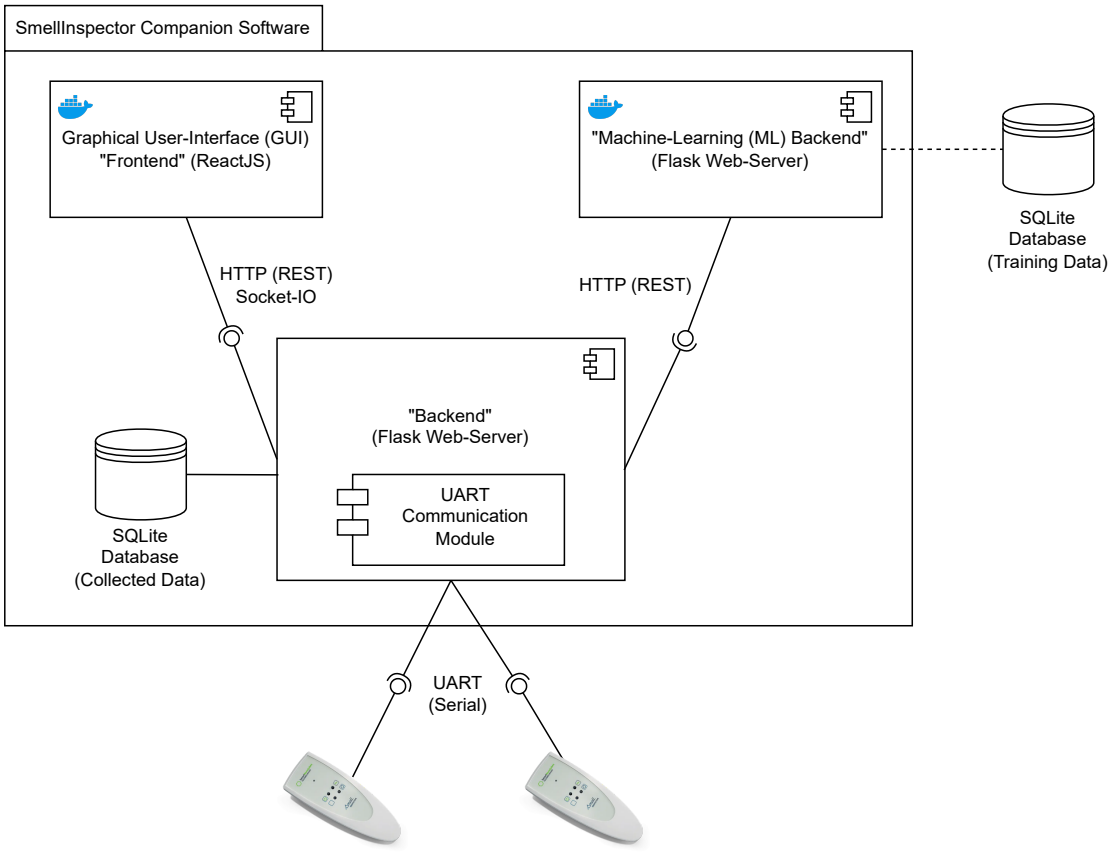


Figure 4.1: The SmellInspector-Companion Software's Architecture

RF classifier was used alongside a KNN classifier to validate SmartNanotubes' claims regarding pattern-based substance classification. An XGBoost classifier was also evaluated experimentally but later discarded.
The following sections describe each of the above-mentioned software components in detail.

4.2 The "Backend" Component

After careful consideration, the "Backend" component was developed as a Python-based REST service to enable multiple users to access SmellInspector data streams in real time across different devices and operating systems. This design allows multiple users to interact with connected devices simultaneously and provides the flexibility to capture live data from inside an incubator without requiring physical proximity. Additionally, the REST service supports the development of various automations by exposing its endpoints, enabling the creation of real-time alarm systems, gas classification dashboards, and other related applications.
To support this functionality, the "Backend" was divided into several sub-components:

1. The REST-Controller component: Provides the REST service endpoints for other applications to communicate with the backend, submit and retrieve data, and supports software expansion through third-party modules.

2. The Serial Communication Service component: Facilitates communication with connected SmellInspector devices via serial UART. When devices are connected to the host machine, this component also enables one-click connection using a custom port scanner, as described in 4.2.1.

3. The Database Service component: Stores data collected from SmellInspector devices for later use, archiving it under common test names along with corresponding metadata, including test date, substances used, test duration, and the specific SmellInspector device employed.

4. The Test Handler component: Allows independent communication with each connected SmellInspector device and is dynamically created whenever a new test is started.

The following diagram 4.2 illustrates the relationships between these components. For each test, a separate Test Handler component is initialized, which uses the UART protocol to poll data from the connected SmellInspector device. The collected data is then stored in the dataset corresponding to the current test.
The REST component retrieves its data from both the Serial Communication Service and the Test Handler components, providing direct access to data from ongoing tests as well as previously archived datasets.

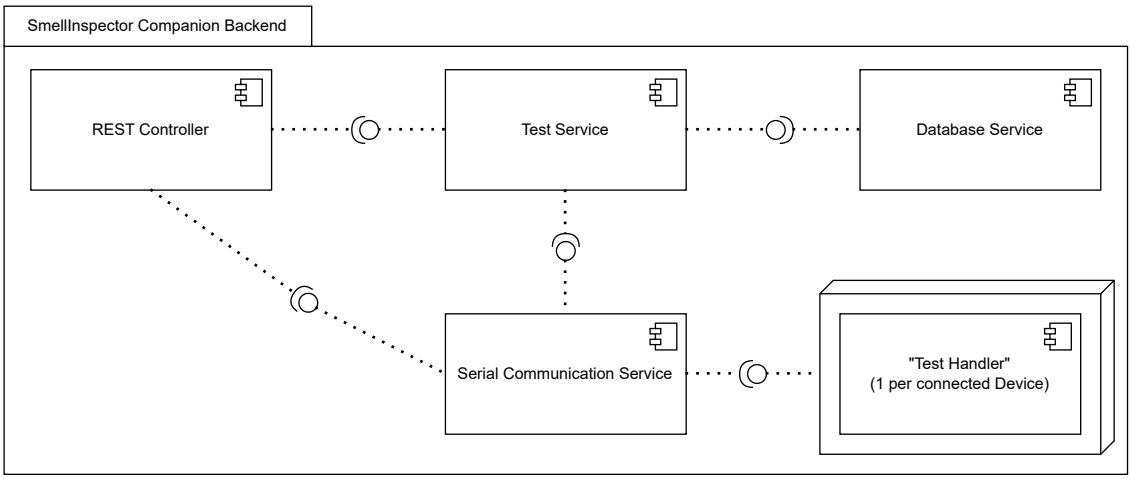


Figure 4.2: The Backend's Architecture

4.2.1 UART (Serial) Communication

The SmellInspector devices use a UART-to-USB bridge for data transfer and begin transmitting data immediately after receiving power, regardless of whether a host device is connected. Sensor data is transmitted at a baud rate of 115200, with each transmission taking approximately 1.8 seconds, as per specification [17]. The data is encoded in simple ASCII bytes, allowing it to be read directly as plain text without additional decoding.
Sending data to the device is possible at any time. However, since the device does not support a full-duplex connection, data received immediately after sending a command

may contain lost bytes. Due to the absence of error correction in the transmission protocol, these incomplete packages are discarded. Because transmissions from the backend to the device occur only during connection/disconnection events and sensor configuration, this approach does not affect active tests and collected experimental data.
The current version of the SmellInspector allows issuing the following two commands:

1. *GET_INFO*: Returns the device's hardware information, including the MAC address (used for device identification in the software) and the firmware version.
2. *FAN0, FAN1, FAN2, FAN3*: Sets the internal fan speed to one of the predefined profiles, with FAN0 turning the fan off completely. Since the fan operates at state 2 by default, this setting was not incorporated into the software. Running the fan at maximum speed would reduce its lifespan, while lower speeds, when tested, further decreased the sensor's response to presented substances.

When reading data from the device, the transmission begins with the word "start," followed by 64 values representing electrical resistance in Ohms. Each value corresponds to one nanotube, resulting in a total of 64 "channels" (4 smell.iX16 sensors × 16 channels per sensor). In addition to the resistance values, one temperature and one humidity reading from the device's hygrometer and thermometer are included. To ensure the accuracy of temperature and humidity measurements, an independent hygrometer and thermostat were positioned next to the sensor at all times.
The 64 resistance values are divided into four sections, as each set of 16 values corresponds to one of the four smell.iX16 sensor arrays within the SmellInspector. In their software, SmartNanotubes calculated and displayed the average resistance for each channel across all four sensors [2, 3]. Table 4.3 showcases the 16 channel values for each of the four sensors that are retrieved with every poll. The "Detector Type" column is provided for reference only and is not included in the actual data.

Detector Type	Ch1	Ch2	Ch3	Ch4	Ch5	Ch6	Ch7	Ch8	Ch9	Ch10	Ch11	Ch12	Ch13	Ch14	Ch15	Ch16
Type1	999	999	11	11	11	3	3	3	2	2	2	1	1	1	999	999
Type2	999	999	999	999	999	9	9	9	6	6	6	5	5	5	999	999
Type3	999	999	14	14	14	10	10	10	7	7	7	4	4	4	999	999
Type4	999	999	15	15	15	13	13	13	12	12	12	8	8	8	999	999

#Note: Ignore channel value with 999 in the measurement.

Figure 4.3: Table Showcasing 16 Channels for each of the four iX16 Sensor

Although the table indicates that values of 999 should be ignored, testing revealed that faulty sensor readings from degraded sensors could still produce seemingly "valid" data not flagged as 999. These faulty values were identified by their thousandfold deviation from the average resistance readings. While normal sensor values are reported in kiloohms, the defective readings appeared in the megaohm range (See Section 3.4).
When a test is started in the Frontend, sensor data is automatically polled every 1.8 seconds, and the raw data is stored in the application's database. To preserve the temporal component, samples are recorded in sequential order, starting with the first sample of the test run and ending with the last. Because the polling interval can vary slightly, the Frontend reports sample positions as "sample counts" rather than actual time in seconds. For evaluating sensor performance, this approach does not introduce bias, as the sensor arrays have a response time of several seconds [17].

4.3 Frontend

To enable non-technical users to operate the sensor in clinical and research settings, a minimalistic web-based user interface was developed. In contrast to the SmellAnnotator software from SmartNanotubes, which presents extensive technical data in complex formats that can overwhelm users, the custom interface was designed to conceal advanced functionalities behind simple, easily navigable screens.

4.3.1 Dashboard

The interface can be accessed from any modern web browser, providing a platform-independent user experience. Upon opening the application, the user is presented with the "Dashboard" page 4.4.
From the Dashboard, the user can navigate between several key functionalities: managing connected SmellInspector devices, organizing the substances available for classification and measurement, and initiating tests for both data acquisition and live inference.

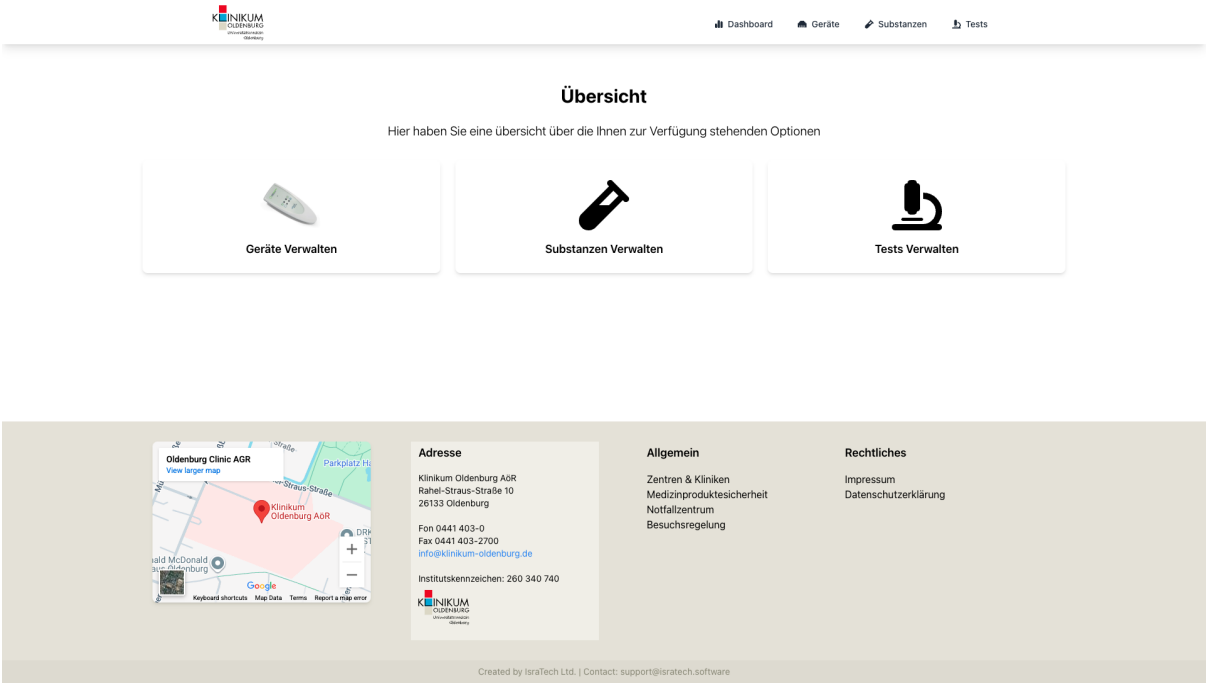


Figure 4.4: The Dashboard page

4.3.2 Device Overview

Clicking on "Manage Devices" navigates the user to the device overview page 4.5. On this page, the user can see all currently connected devices, review previously connected devices, and add new SmellInspector devices to the system.

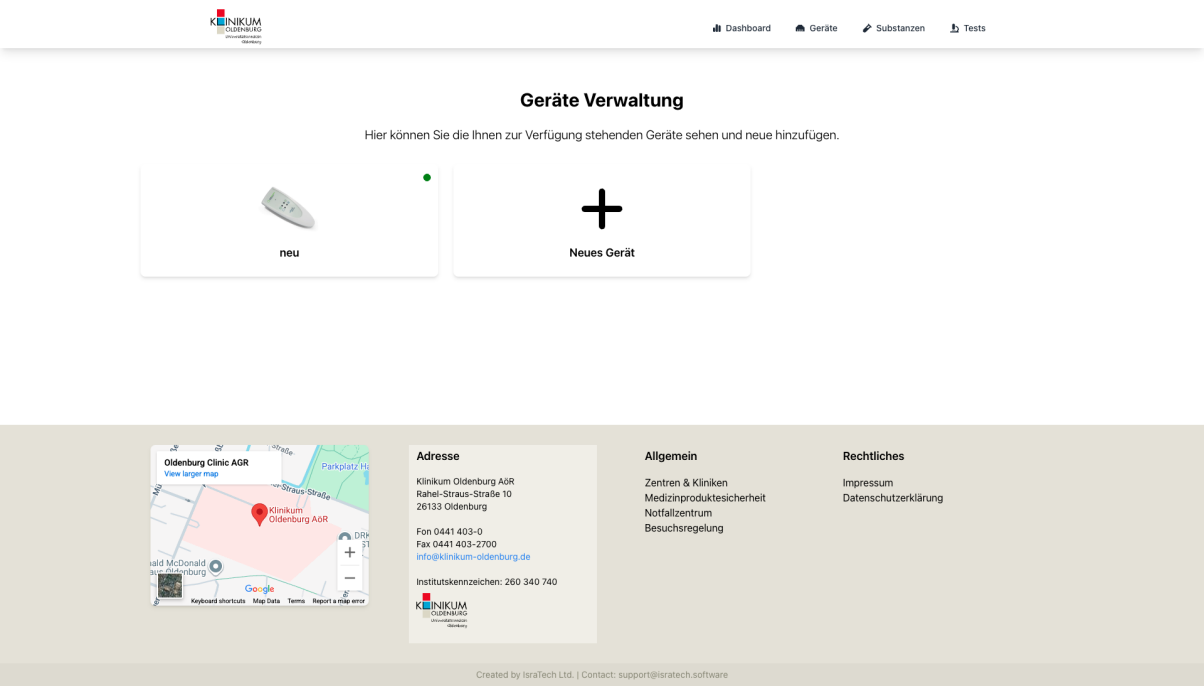


Figure 4.5: Device overview page

4.3.3 Device Management

Clicking on any listed device navigates the user to that device's management page 4.6.

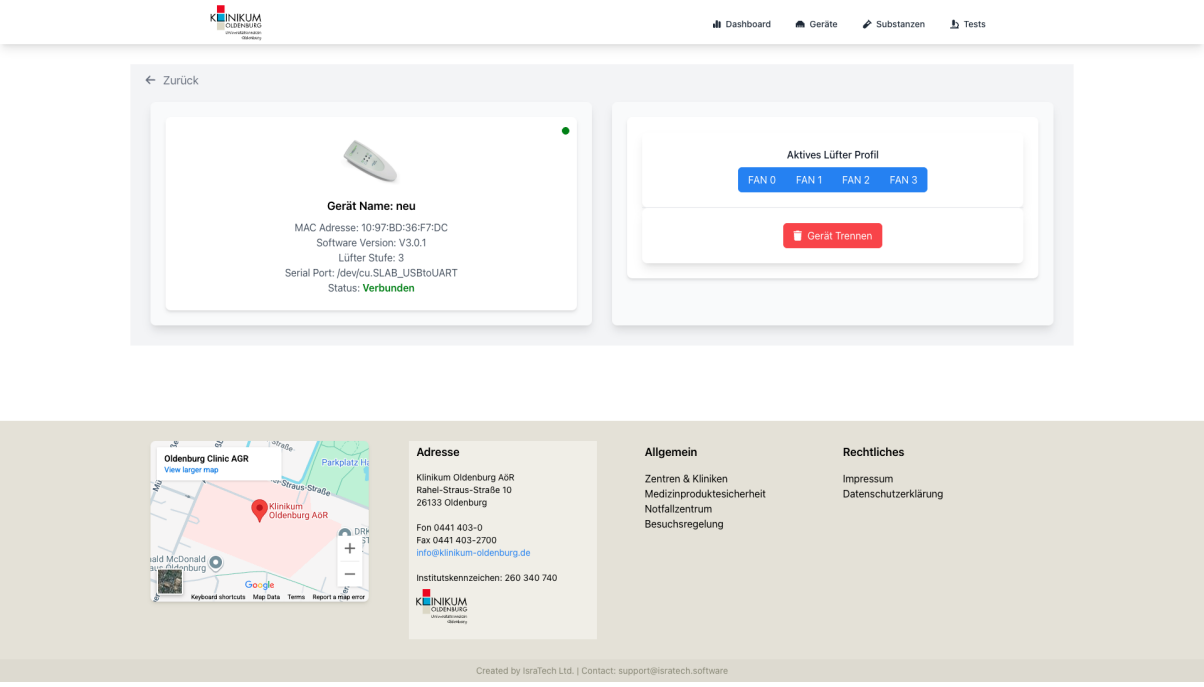


Figure 4.6: Device management page

Here, the user can view an overview of the SmellInspector's hardware information:

- **MAC Address**: The device's MAC address, used to uniquely identify it within the software, allowing multiple devices to operate in parallel.

- **Software Version**: The firmware version of the device, important for ensuring accurate data acquisition.
- **Fan Speed**: The speed of the intake fan, which directs air onto the smell.iX16 sensor arrays.
- **Serial Port**: The hardware port to which the device is connected on the host device.
- **Status**: Indicates whether the sensor is currently connected or disconnected.

When clicking on a disconnected device, indicated by a red dot and a gray background, the user is presented with the same interface, but with an option to reconnect the device 4.7. The reconnection process is analogous to the initial connection of the device to the software and is described in Section 4.3.4.

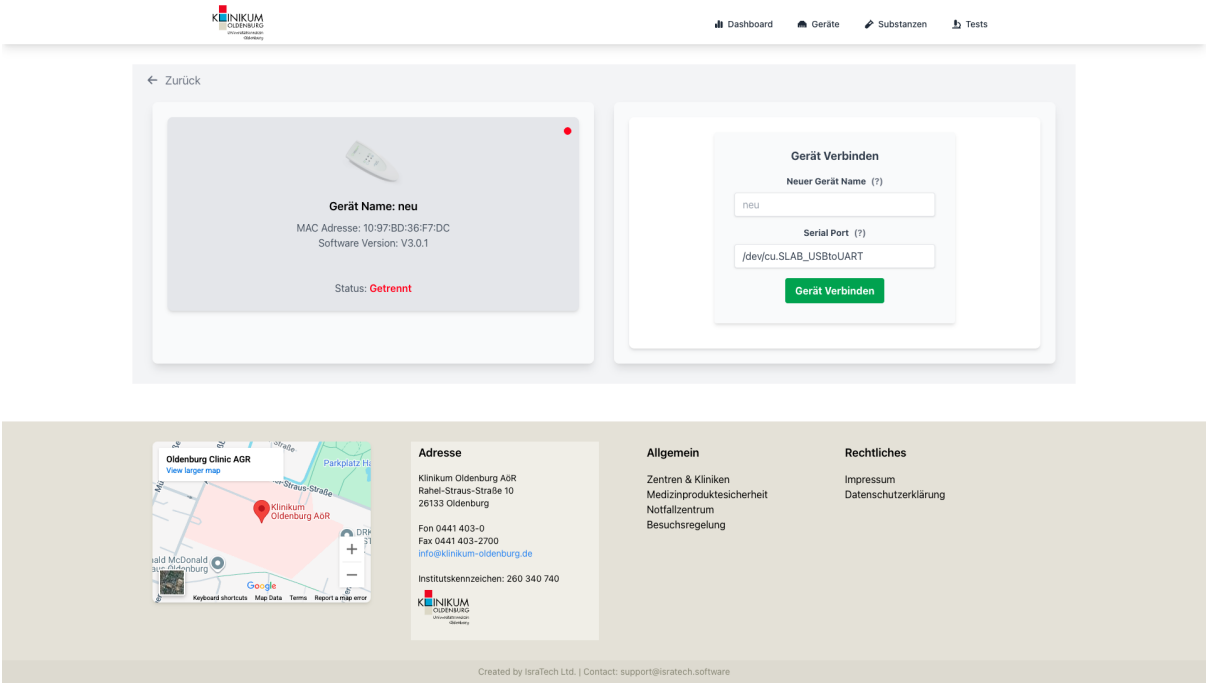


Figure 4.7: Device management page for a disconnected device

4.3.4 Adding a new device

Upon first launch, the user navigates to the device overview page 4.3.2 and clicks the card with the plus icon. This opens the page for adding a new device 4.8.
The SmellInspector supports two methods for establishing serial communication:

1. Connecting over Bluetooth and decoding raw data from the Bluetooth protocol.
2. Reading data through a USB-to-UART bridge connection.

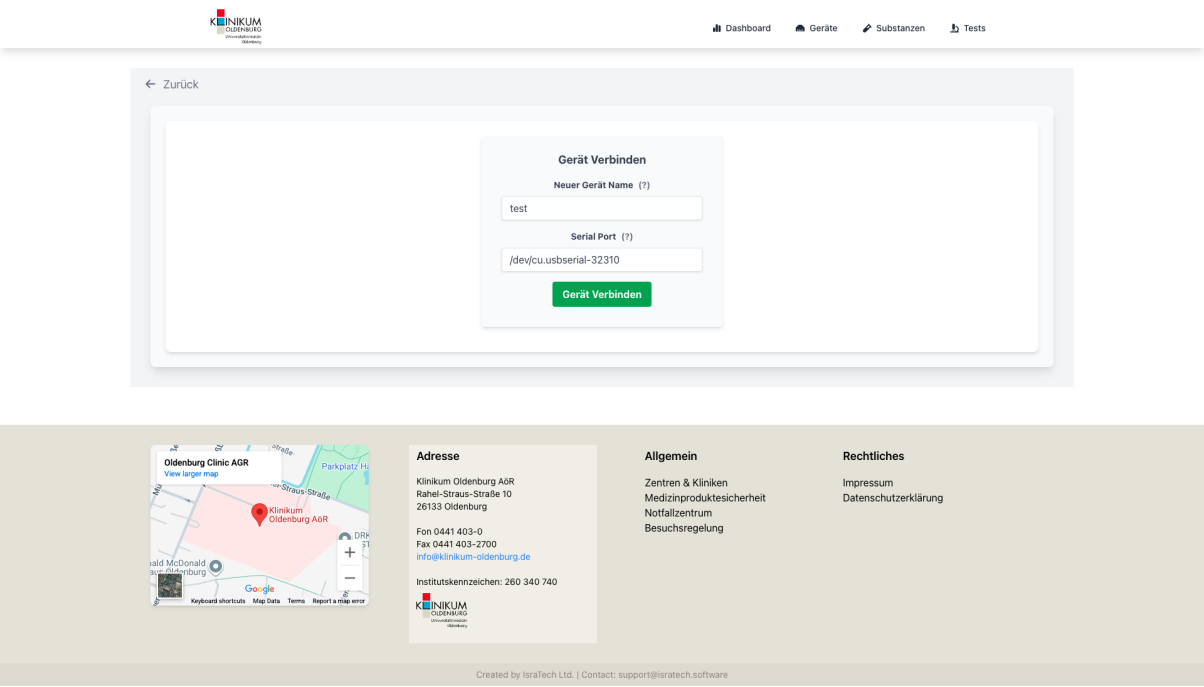


Figure 4.8: Add new device page

To ensure cross-platform compatibility, the software uses a serial connection for reading data from the SmellInspector. To enable this, the host running the backend software must have the CP210x UART-to-USB driver installed. If the driver is not installed, the user is presented with an error message on this page, and no devices can be connected.
If the driver is installed, an automated port scan is performed to detect connected SmellInspector devices, which are then displayed in a drop-down list.
To connect a device, the user selects the port to which the SmellInspector is connected from the drop-down menu and clicks “Connect device.” Upon successful connection, the device’s information is read and becomes accessible on its management page 4.3.3.
When a device is disconnected and later reconnected, the same port scan process is executed. The software identifies SmellInspector devices using their MAC addresses, preventing the same device from being connected multiple times.

4.3.5 Substance Management

After connecting the SmellInspector devices, the user navigates to the substance management page 4.9 to view existing substances and add new ones before starting a test.
Each substance includes a name and a quantity attribute, enabling the classification of both the substance names and quantities [1].

[1] According to specification [17], the SmellInspector is unable to classify substance quantities. The exploratory experiment results from Section 5.4 suggest that classification could be possible, if data is manipulated correctly.

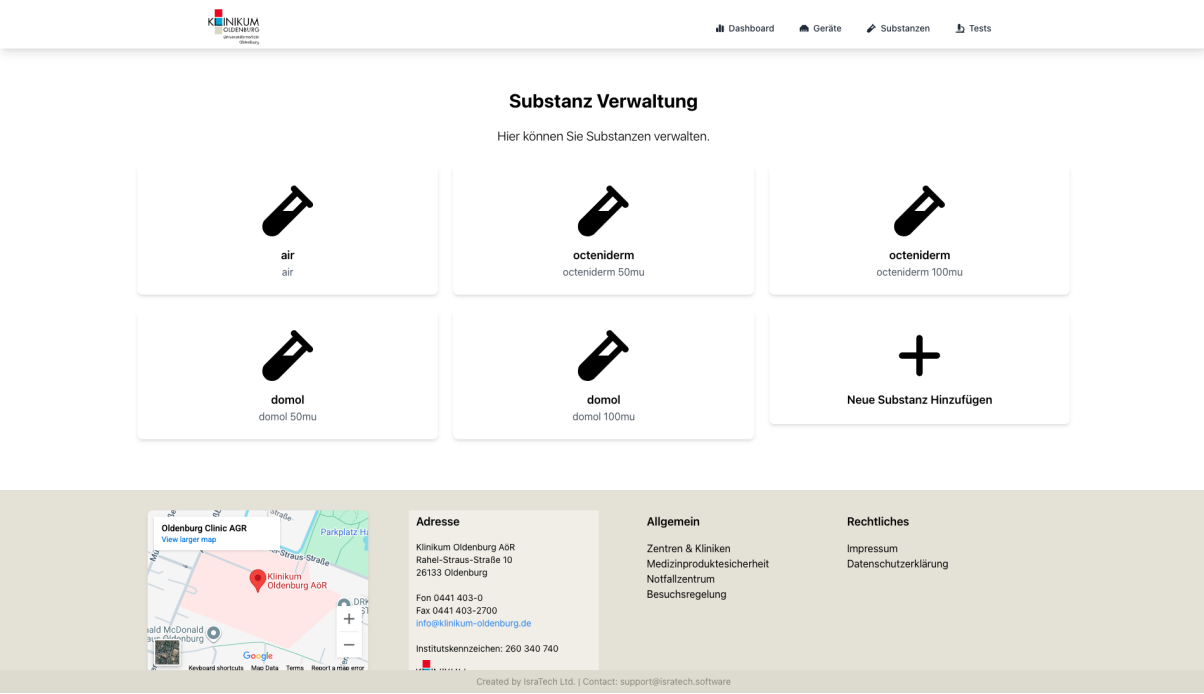


Figure 4.9: Substance management screen

To add a new substance, the user can click the card with the plus icon, navigating them to the page for adding new substances, as seen below 4.10.

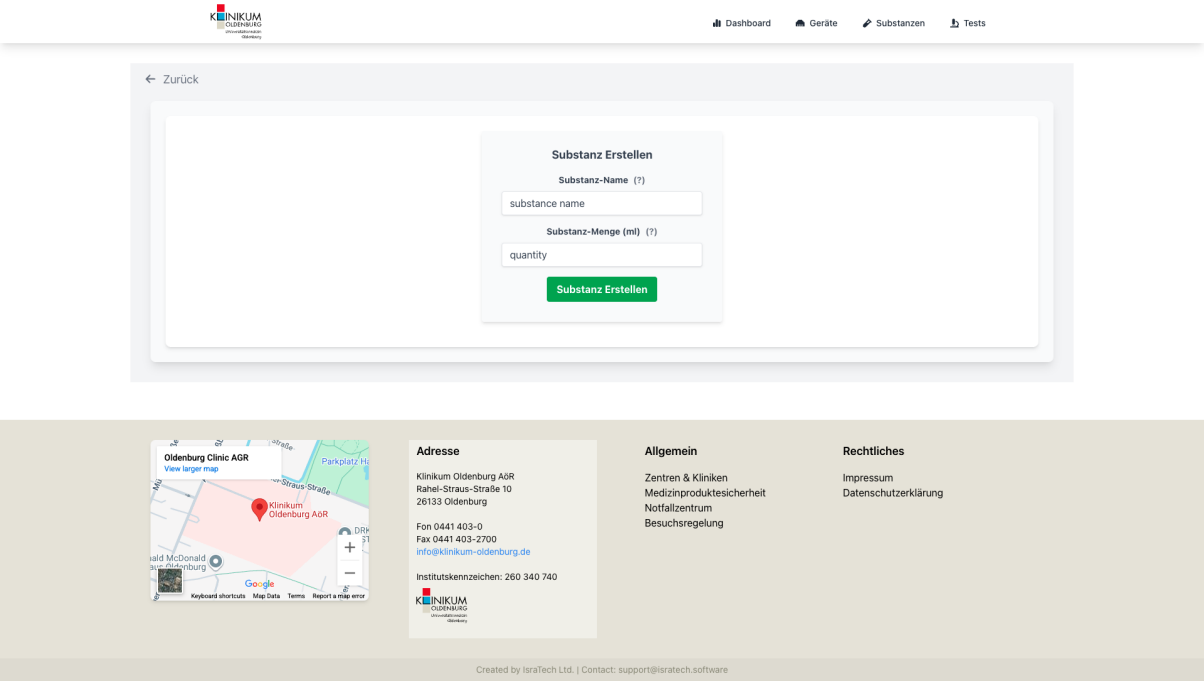


Figure 4.10: New substance page

4.3.6 Test Management

After configuring their SmellInspector device and substances, the user can start a new test instance.
In this context, a "test instance" refers to the software actively collecting data from a connected SmellInspector device. Even when performing live inference, a test instance must be started in order to view classified substances.
Optionally, the software can save the collected data under a common identifier, the test's "name", allowing it to be referenced later. Enabling this option creates "labeled" data points, where each data point is associated with the currently measured substance. This feature allows users to efficiently generate large amounts of training data for machine learning models (ML) without manually labeling each data point. When data collection is enabled, ML models are automatically re-trained for every 100 collected data points, providing both real-time inference and continuous improvement of model detection accuracy.
Note: The test mode is intended to be used either for generating training data or for performing live inference. When training data is collected, ML models are re-trained as a proof of concept for a real-time data acquisition and ML model re-training feedback loop. To perform inference only, without retraining the models, the data collection option should be disabled when starting a test.
A new test instance can be initiated through the test overview page 4.11, which provides an overview of all active test instances. Hereby each test instance corresponds to a single SmellInspector device. A single device cannot be used by more than one test instance at a time.

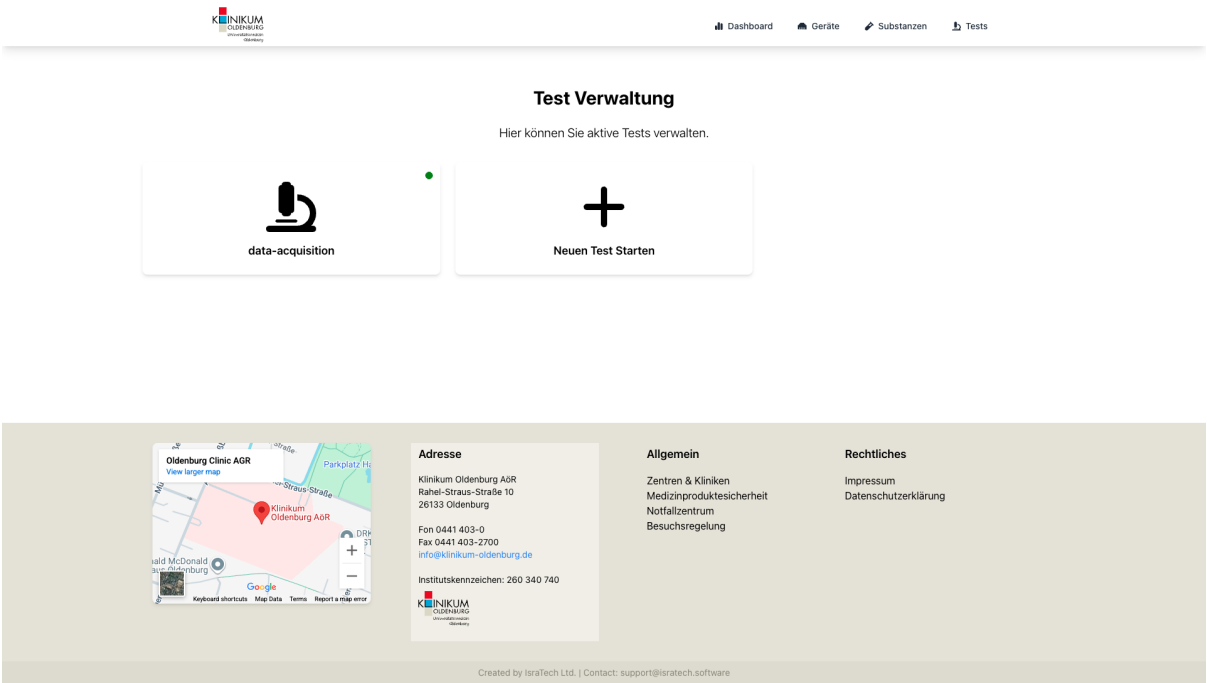


Figure 4.11: Test overview page

Starting a new test follows a process similar to adding new substances: the user clicks the card with the plus icon and enters a test name. They can then select the SmellInspector device to use and choose whether the collected data should be stored in the application's database as training data. The test is initiated by clicking "Start test", as shown in Figure 4.12.

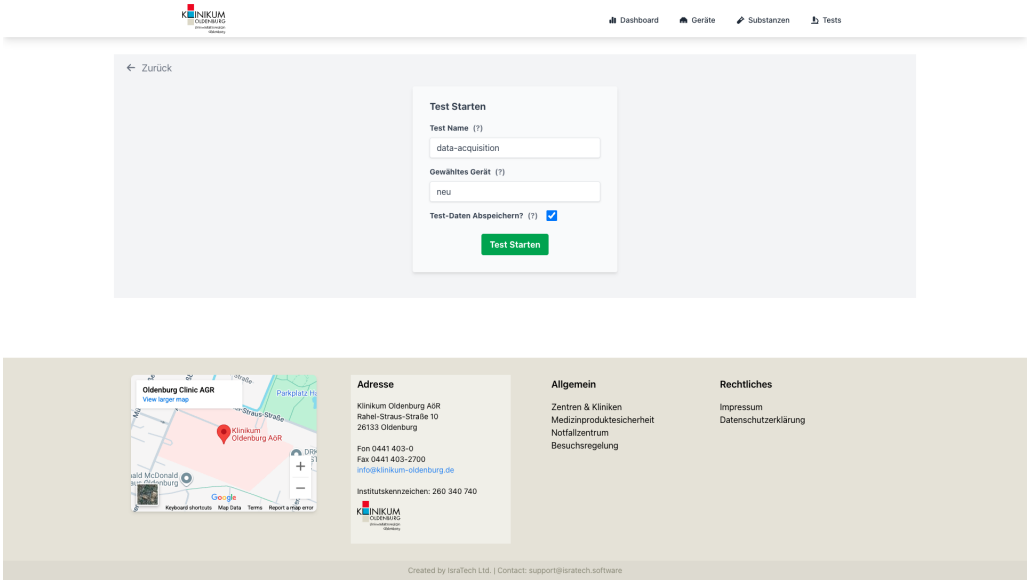


Figure 4.12: New test page

Once a test has been started, the user can click on its card to navigate to the test details page 4.4.

4.4 Test details page

On the test details page 4.13, the user has access to a live data feed from the connected SmellInspector device. The interface is organized into four panes:

1. **Top-left pane:** Displays information about the SmellInspector device, including its hardware and software details, the raw data collected from the sensor (including temperature and humidity), and the classification results from enabled machine learning (ML) models.
   For the machine learning models, abbreviations are used to save space: "RF" stands for "Random Forest", and "KNN" for "K-Nearest-Neighbour".

2. **Top-right pane:** Shows the currently selected test substance and allows the user to change the measured substance. When data collection is enabled, the selected substance is added to the raw sensor data as a label for ML models. Using the "Change measured substance" button automatically reverts the dropdown to the previously measured substance, enabling single-click cycling between two substances without reselecting each time.

3. **Bottom-left pane:** Graphically renders the raw data from the SmellInspector's four smell.iX16 sensor chips. Each iX16 sensor outputs 16 values, resulting in a total of 64 linear plots.
   Labels for each plot are provided in the legend below the pane.

4. **Bottom-right pane:** Displays the SmellInspector's data as rendered by Smart-Nanotubes' SmellAnnotator software. For each of the 16 channels on a smell.iX16 chip, values across the four sensors are averaged, producing a single plot per channel, for a total of 16 plots (across 4 sensors).
This graphic is included as a reference to the SmellAnnotator software. Although sensor values vary significantly across the four sensors (see Table 5.19), making the averaged values less useful, it was added to maintain consistency across software applications and allow comparison of results.

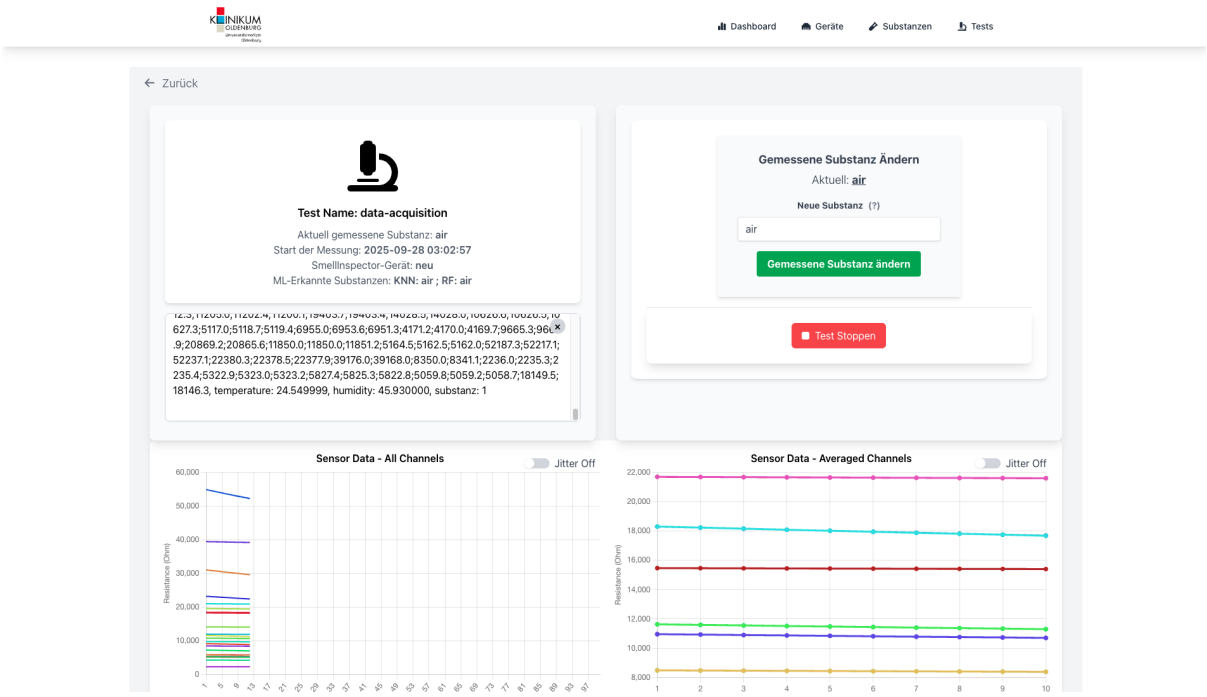


Figure 4.13: Test details page

4.5 Machine Learning

To maintain the software's modular design, the machine learning components were implemented as a separate module, completely independent from the backend component. Figure 4.14 illustrates the abstract architecture of the connected machine learning adapters, which are described in detail in the following Section.

4.5.1 ML-Adapters

To allow rapid prototyping with different machine learning (ML) algorithms, each ML model is encapsulated within its own component. The "ML Handler" component manages the connection of multiple models to the active classification pipeline.
Each ML component implements the "MLAdapter" interface, which exposes three public methods:

- *fit(data, labels)*: Trains the machine learning model using the provided data and labels.
- *predict(data)*: Performs inference on new data to predict its corresponding label.
- *classes()*: Returns a list of available labels for the model.

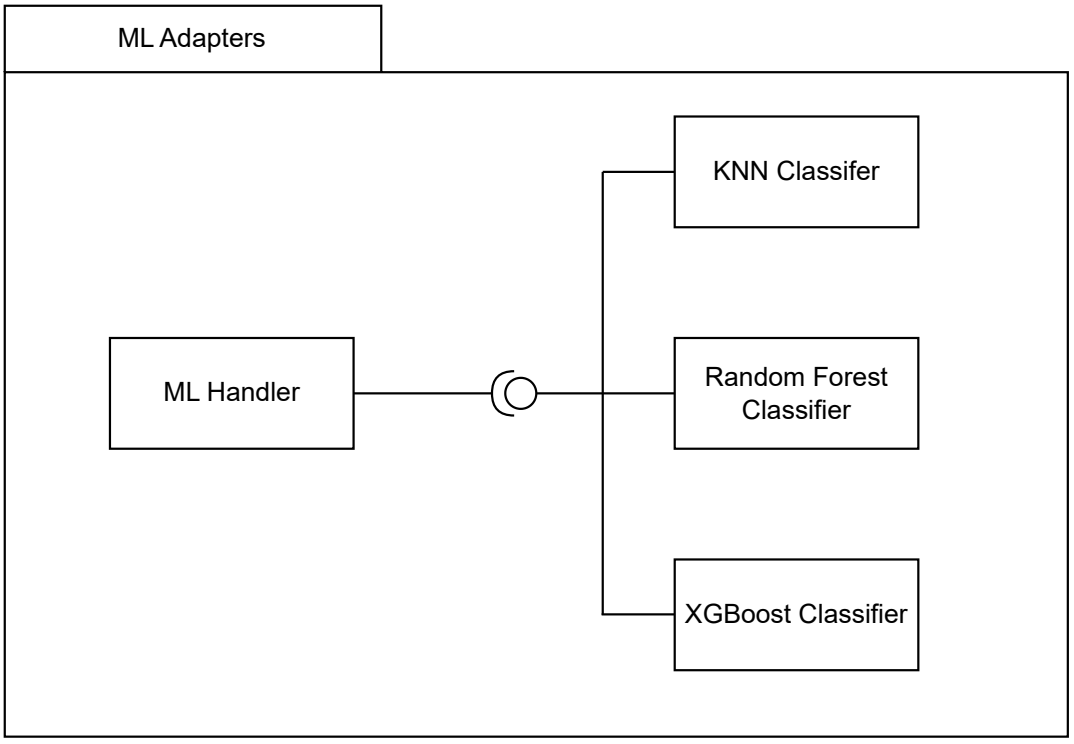


Figure 4.14: The machine learning component's architecture

By default, two classification models are enabled: a KNN model and a RF model. These models provide baseline performance for substance classification and can be used immediately without additional configuration.

4.5.2 Model Training and Continuous Re-Training

As the machine learning component operates as standalone software, it does not have access to data collected from previous tests and therefore cannot perform any classification immediately after startup.

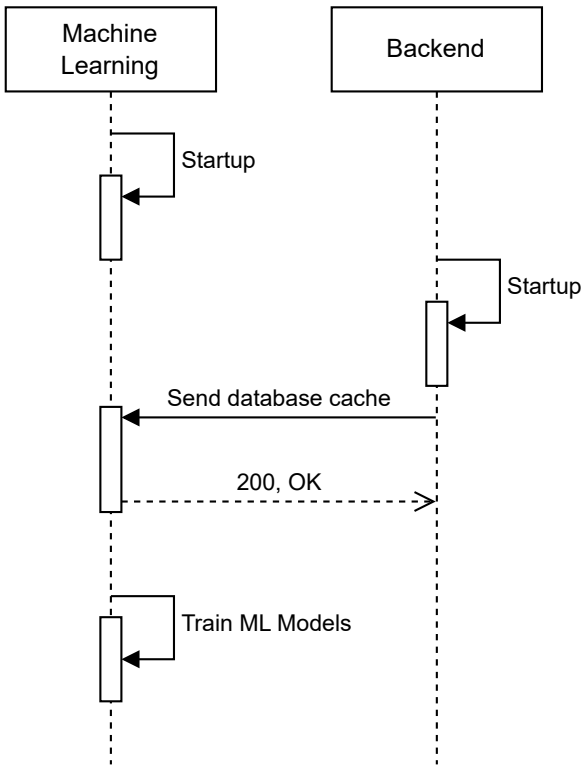


Figure 4.15: Initial ML-Model Training Sequence Diagram

To enable model training, the ML component is started alongside the Frontend component, before the Backend component, ensuring it is ready to receive both historical and new data from the Backend. Once the Backend is running, a copy of its database is sent to the ML component, allowing it to train the enabled ML models using previously collected data. This synchronization process runs automatically when the components are started together and only needs to be executed once. Figure 4.15 illustrates the initial training process.
After training, the ML component can provide real-time classification results from the enabled models through the *"/predict"* endpoint.
To allow continuous learning from newly acquired data, the component also exposes a *"/new"* endpoint. This endpoint can be called by the Backend component to supply new data as it is collected from the connected sensor(s). This feature can be enabled or disabled when starting a new test via the interface (see Section 4.12). When enabled, the ML models are automatically retrained for every 100 collected samples, as illustrated in the re-training sequence diagram 4.16.

The retraining frequency can be adjusted dynamically, a process explained further in Appendix 7.1.

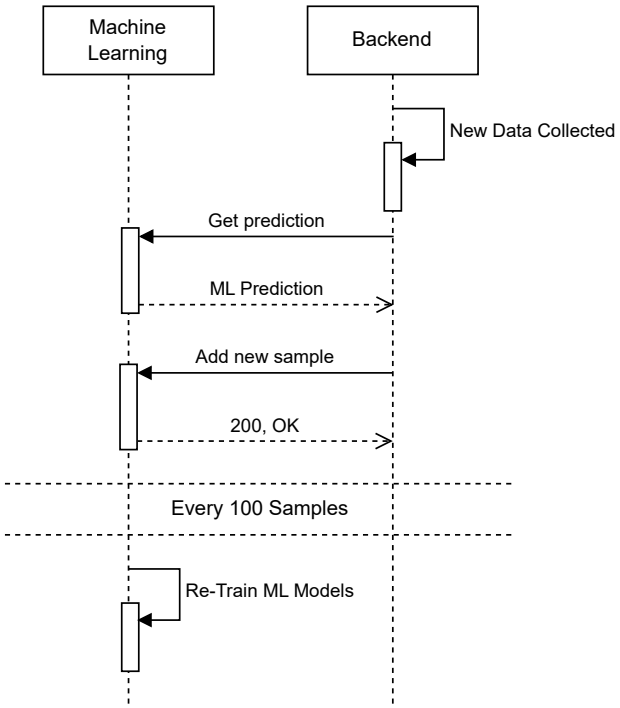


Figure 4.16: ML-Model Re-Training Sequence Diagram

4.5.3 Data Manipulation

In addition to continuous retraining, the ML component provides a range of pre-processing and data manipulation tools to improve ML model training:

- **Data scaling:** Supports SciKit-Learn scalers to normalize and transform data before feeding it into enabled ML models. Currently supported scalers are the "StandardScaler" (scales data to a mean of zero and a standard deviation of one) and the "RobustScaler" (provides outlier-resistant scaling).
- **Automatic n-estimator computation (Random Forest):** Automatically selects the number of trees for the Random Forest (RF) classifier using Out-of-Bag (OOB) scoring. The default range is 100 to 1000 trees, which can be dynamically adjusted. This feature can also be disabled via the component's configuration file (See Appendix 7.1).
- **Sample re-labelling:** Attempts to re-label non-air samples that were incorrectly labeled. A detailed explanation of this algorithm is provided in Section 5.3.
- **Training data selection:** The Backend stores test data across all recorded test instances, which may span days or weeks. To prevent overfitting and account for

changing environmental conditions, the ML component allows selection of training data based on test names, limiting the training data to chosen test instances. Additionally, separate test instances can be used as validation or test data during initial training to perform cross-validation.

- **Windowing/sample grouping:** Since test data is recorded as a continuous stream of independent samples, time-series correlations are captured by grouping samples into windows during training and testing.
- **Data balancing and artificial oversampling:** Allows creation of balanced or unbalanced datasets by artificially increasing or decreasing the number of available samples.

Further details on sample grouping, artificial over- and undersampling, and re-labeling are provided in Chapter 5. These data manipulation tools were implemented to enable a wide range of experiments and analyses using the recorded data.

4.5.4 Real-Time Inference

Finally, to enable real-time classification of currently measured sensor data, the ML component provides the *"/predict"* endpoint. This endpoint receives raw data from connected sensors and returns a list of predictions from each enabled ML model. The predictions are then displayed in the frontend during a test instance within the test description, as illustrated in the image below.

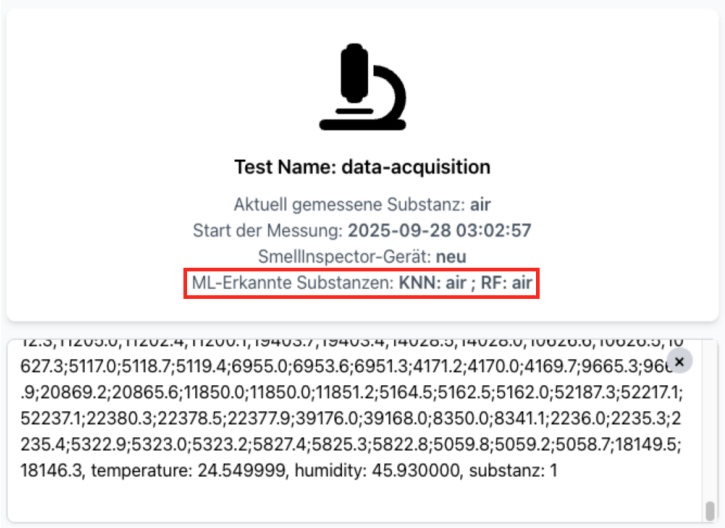


Figure 4.17: Live inference results as seen in Frontend

For each model, an abbreviation is given to save space in the text box. For the currently enabled models, *"KNN"* stands for *"K-Nearest Neighbour"* and *"RF"* stands for *"Random Forest"*.

Chapter 5

Results and Evaluation

Author's note: Measurements were reapeated over several months with continuous evaluation of intermediate results. During this time, various changes were made to the data acquisition mechanism and the test procedure, as discussed in Chapter 3. As a result, the evaluation in this chapter is concerned with the latest results from August through October 2025 and earlier data is omitted.
To evaluate the performance of the SmellInspector for real-time, in-situ substance classification, two machine learning (ML) approaches were implemented: a Random Forest (RF) classifier for actual substance classification, and a K-Nearest Neighbour (KNN) classifier for comparison.

5.1 Single-Sample Classification

In the absence of prior reference data, raw sensor outputs were initially fed directly to the classifiers without any pre-processing. This approach produced unrealistically high performance, with reported F-scores reaching 95%. The method trained on isolated samples and did not account for temporal dependencies, resulting in temporal leakage where correlated samples appeared in both training and test sets.
As a result, the data was restructured, grouped, and pre-processed to properly account for temporal dependencies, as detailed in the following section.

5.2 Data Preparation

To account for differences in resistance scaling across channels, sensor data from all 64 channels was normalized using a "StandardScaler" in Python. To capture temporal correlations within the time series, the data was divided into "Sample Groups", with each group representing a time window during the measurement and sharing a common substance label. This grouping procedure is illustrated in Figure 5.1. For training, the sample groups were shuffled to introduce randomness and ensure variability in the model training process.

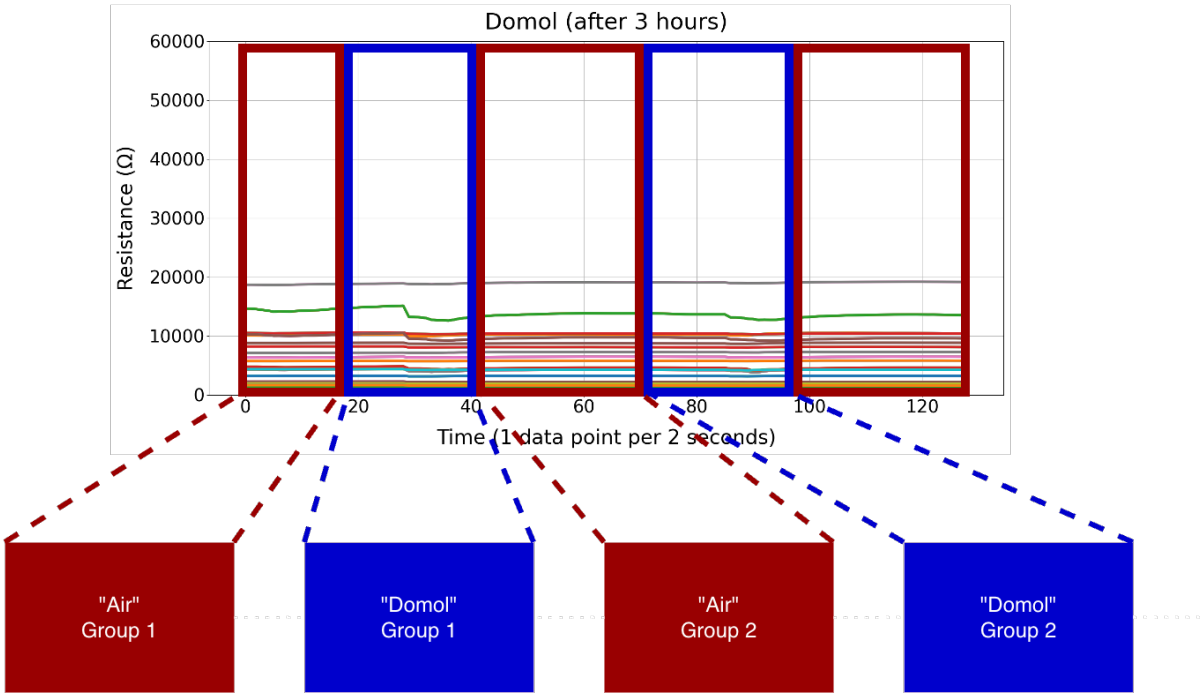


Figure 5.1: Sample Grouping of Test Data

For each unique label, 70% of the corresponding sample groups were allocated for training, while the remaining 30% were reserved for validation, as illustrated in Figure 5.2. This procedure was applied both to data obtained from a single continuous test run conducted over several hours and to datasets collected from separate experiments performed on different days.

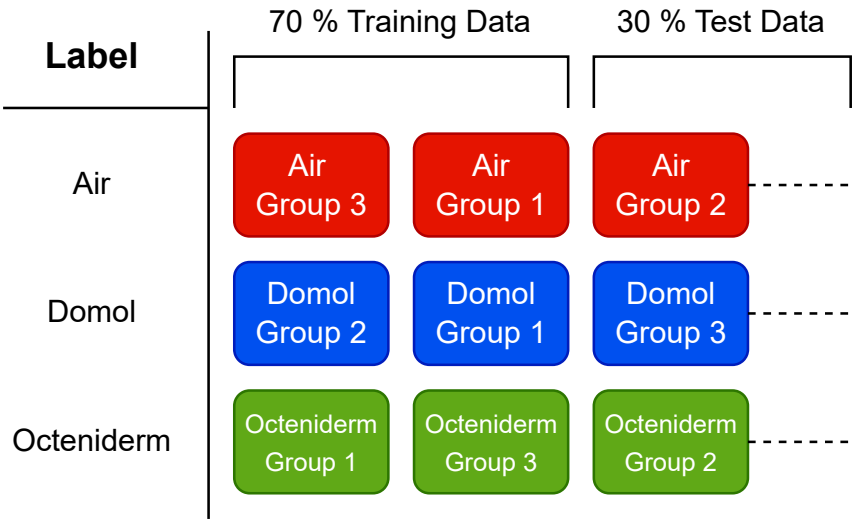


Figure 5.2: Train / Test Split of Shuffled Sample Groups

Regarding label selection, the initial classification trials used only the substance names as labels. This approach was intended to first assess the sensor's ability to distinguish

between different substances, following the goals outlined in Chapter 1.
In subsequent experiments, substance quantities were incorporated into the labeling scheme, with the corresponding results discussed in Section 5.4.

5.2.1 Further Adjustments

To address the oversaturation of certain substances in the experimental data, a control mechanism was introduced to allow training with balanced datasets instead of using raw, unevenly distributed data. Under normal operation, the ML component (Section 4.5) performs real-time classifier re-training with newly acquired sensor readings, which can lead to an overrepresentation of "air" samples compared to other substances.
The implemented switch allows users to apply over- and undersampling techniques to adjust the representation of different substances, promoting more balanced and representative classifier training. Undersampling removes random sample groups from both the training and validation datasets until the number of sample groups matches that of the least represented label group. Oversampling duplicates sample groups in both datasets to match the size of the largest label group. In both cases, the procedure is applied independently to the training and validation datasets, preventing data leakage.

5.2.2 Data Re-Labelling

After evaluating the classifiers using recorded test data, an additional source of bias was identified.
During sample introduction, sensor readings were labeled with the target substance name immediately, before the substance had sufficient time to disperse throughout the measurement chamber. Consequently, for each sample group, approximately half of the data points labeled with a substance's name should, in fact, be labeled as "air". This introduces a classification bias and lead to artificially lower classification performances.
To address this, a data re-labeling algorithm was developed. The algorithm processes the time-series data sequentially, starting from the first sample, and adjusts labels based on historical humidity readings recorded by the SmellInspector's hygrometer. Its purpose is to distinguish between samples that are merely labeled with a substance name and those in which the substance is actually present in the ambient air.
To achieve this, the algorithm calculates the mean humidity of the ambient air prior to sample introduction. When the label transitions from "air" to a specific substance, the algorithm determines a threshold slightly above this baseline mean. All data points within the current sample group that exhibit humidity values below this threshold are then re-labeled as "air". This process is repeated for all non-"air" sample groups, with the ambient humidity recalculated between successive measurements to account for gradual environmental changes over time. This procedure is repeated for each subsequent sample group, as outlined in the pseudocode 5.3.
The threshold parameter is dynamically adjustable. For the evaluated datasets, it was defined as the average between the ambient mean humidity and the 75th percentile of the windowed humidity distribution. For example, if the mean ambient humidity prior to sample introduction was 25%, and the 75th percentile of the windowed humidity values was 35%, the resulting threshold would be:

$$threshold = \frac{25 + 35}{2} = 30\%$$

```
threshold = 0
current_sample = air

for sample in all_sample_groups:
    if current_sample != sample.label:
        if current_sample != air:
            current_sample = air
            threshold = 0
        else:
            current_sample = sample.label
    if current_sample != air:
        if sample.humidity < threshold:
            sample.label = air
    else:
        threshold = re_compute_threshold_based_on_mean()
```

Figure 5.3: Re-Labeling Algorithm

Any values below this threshold were re-labeled as “air”, thereby removing incorrectly labeled samples from the substance’s sample group.
To identify an appropriate threshold, further tests were conducted, starting with the 10th percentile and incrementally increasing the percentile in 10% steps. Additionally, the 25th, 50th, and 75th percentiles were evaluated.

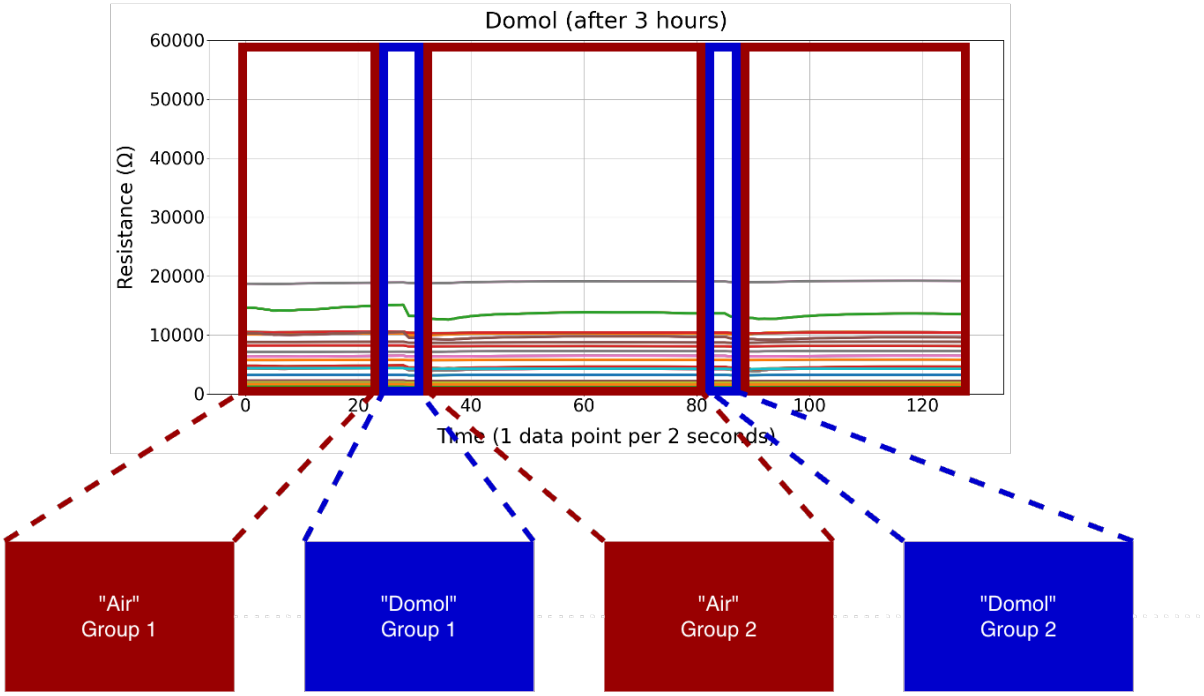


Figure 5.4: Dataset after re-labeling

The 75th percentile provided the greatest improvement in classifier performance and was therefore adopted as the standard threshold for all analyzed datasets.
Figure 5.4 illustrates this for the data from Section 5.1 after performing re-labeling.

5.3 Substance Classification Results

To ensure that the previous data modifications did not introduce unintended bias, toggleable switches were implemented for each modification. To investigate the effects of these adjustments, evaluations were performed separately for each, as detailed in the following sections.
The following sections focus on the evaluation of the most recent dataset, which consists of a total of 12794 samples:

- 6571 "air" samples,
- 3489 "Domol" samples,
- 2734 "Octeniderm" samples.

During oversampling, smaller sample groups were scaled up to match the size of the largest group ("air"). Conversely, during undersampling, larger groups were reduced to match the size of the smallest group ("Octeniderm").
An overview of all classification results can be found in Appendix 7.1.
For all performed evaluations, data was provided to the classifiers as individual samples, each consisting of 64 sensor values with a corresponding output label. The temporal component was incorporated during training through the previously described sample grouping procedure. During validation, models were tested on individual samples drawn from the validation dataset, which preserved the temporal correlations within each sample group.
Performance was assessed using recall and F-score metrics. These metrics were selected because they evaluate both the model's ability to correctly identify instances of each class (recall) and the balance between precision and recall (F-score), making them particularly suitable for multi-class classification. Scores were computed on a per-label basis, treating each label group as a separate class instance, to provide a detailed view of the models' ability to recognize and generalize across all measured substances.
Note: The metrics mentioned in the text always correspond to those of the Random Forest (RF) classifier. The KNN classification scores are included as a reference only.

5.3.1 Unbalanced, No Re-Labelling

To establish a baseline evaluation without additional modifications, normalized test data was initially provided to the classifiers without further adjustments, simulating the conditions of real-time re-training as implemented by the ML component (Section 4.5).
Table 5.1 summarizes the training and test sample distributions for the this evaluation.

Group / Substance	Air	Domol	Octeniderm
Training	4890	2384	1925
Test	1681	1105	809

Table 5.1: Train / Test Sample Distributions

During testing, both machine learning models struggled to correctly classify all three substances. The Random Forest (RF) classifier achieved an F-score of 14.7% for "air" samples and 28.7% for "Domol". With "Octeniderm" dominating the non-"air" substance space, it achieved a slightly higher F-score of 36% (see Table 5.2).
When evaluating detection across non-"air" substances (Figure 5.5), the models successfully distinguished between "Domol" and "Octeniderm", indicating fundamental differences in signal distributions for each substance.

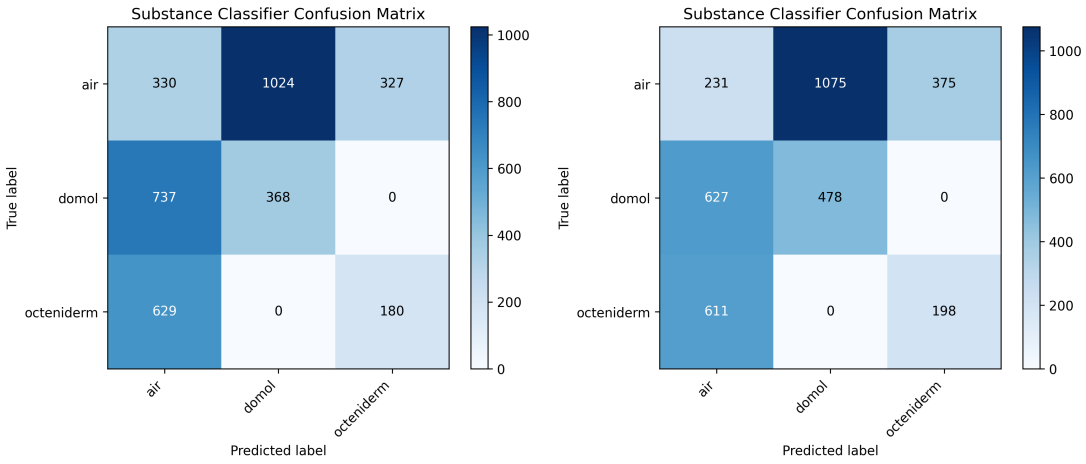


Figure 5.5: KNN and Random Forest Confusion Matrices

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	19.6%	19.5%	13.7%	14.7%
Octeniderm	33.3%	29.5%	43.3%	36.0%
Domol	22.2%	27.4%	24.5%	28.7%

Table 5.2: Statistics for raw, unbalanced data

From initial testing, this suggest that, although "air" samples dominated the training data, potentially biasing the models toward air classification, the differences among the three substances were insufficiently pronounced for accurate classification. To investigate this hypothesis further, the tests were repeated using balanced datasets, as discussed in the following section.

5.3.2 Undersampled, No Re-Labelling

To mitigate the oversaturation of "air" samples, the dataset was re-balanced toward the smallest sample group, and the experiment was repeated.
For undersampling, the data was first divided into training and test sets. After the split, samples from the dominant classes ("air" and "Octeniderm") were randomly removed to reduce their sizes to match that of the "Domol" class. The resulting sample distributions are presented in Table 5.3.

Group / Substance	Air	Domol	Octeniderm
Training	1925	1925	1925
Test	809	809	809

Table 5.3: Train / Test Sample Distributions

Upon testing, detection of non-"air" samples improved slightly, with "Octeniderm" achieving an F-score of 38.7% and "Domol" an F-score of 38.8%. In contrast, classification of "air" samples dropped substantially, reaching only 8.3%.

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	11.6%	9.4%	10.0%	8.3%
Octeniderm	33.9%	34.1%	39.7%	38.7%
Domol	29.9%	39.0%	30.0%	38.8%

Table 5.4: Statistics for undersampled, balanced data

Although balancing the dataset enhanced substance classification, the classifiers still failed to reliably distinguish between ambient air and substance samples, as reflected in the confusion matrices (Figure 5.6).

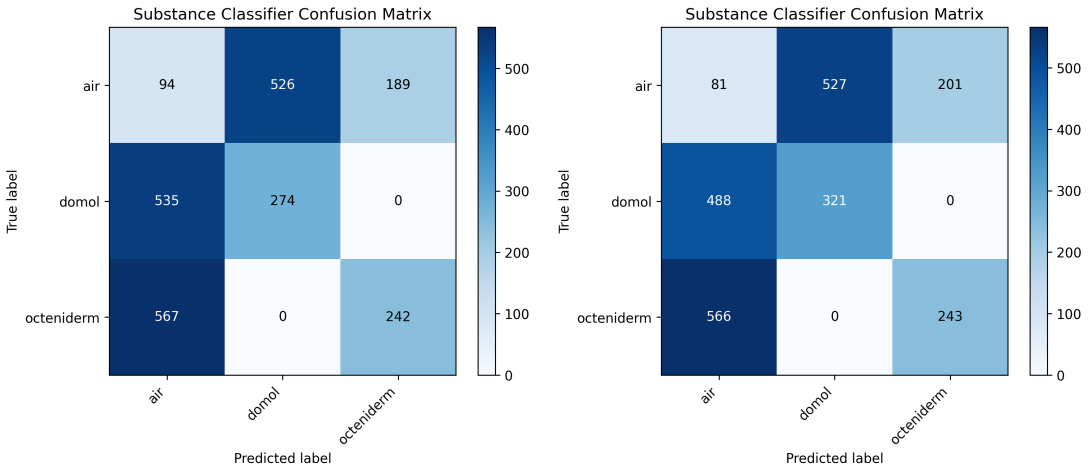


Figure 5.6: KNN and Random Forest Confusion Matrices

To determine whether insufficient data was responsible for these limitations, a final experiment was conducted where non-"air" substance samples were artificially oversampled to achieve equal representation of ambient air and substance classes.

5.3.3 Oversampled, No Re-Labelling

In the final experiment, each non-"air" substance class was repeatedly sampled in both the training and test datasets, resulting in an artificial increase in sample counts for both sets. To avoid introducing bias that could artificially inflate classification accuracy, oversampling was performed only after the data had been split into training and test sets. This ensured that each sample group's size increased independently, preventing leakage of training data into the test set.
In total, 4890 samples per substance were used for training, and 1681 samples for testing, as seen in Table 5.5.

Group / Substance	Air	Domol	Octeniderm
Training	4890	4890	4890
Test	1681	1681	1681

Table 5.5: Train / Test Sample Distributions

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	16.5%	13.1%	14.9%	12.4%
Octeniderm	37.4%	37.4%	46.8%	44.5%
Domol	26.5%	36%	28.3%	37.7%

Table 5.6: Statistics for oversampled, balanced data

When compared to previous results, a modest increase in classification performance was observed across all substance classes. While undersampling had substantially reduced the F-score for "air" samples, oversampling resulted in an F-score of 12.4%. Although the "air" dataset was not directly affected by the oversampling procedure, a slight decrease from the original 14.7% was noted.
For "Domol", detection accuracy decreased slightly by 1.1%, resulting in an F-score of 37.7%. In contrast, "Octeniderm", which was previously the smallest dataset, exhibited a marked improvement, with its F-score increasing by 6.9% to reach 46.8%.

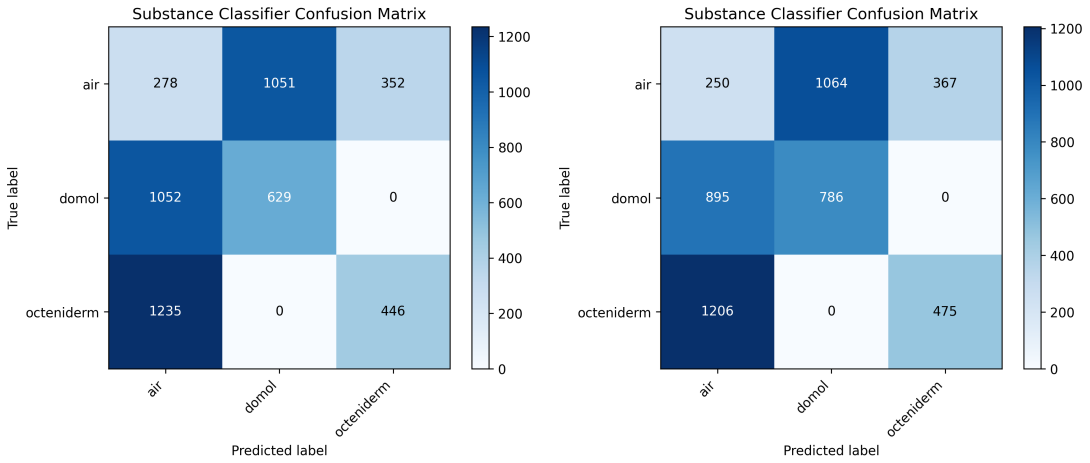


Figure 5.7: KNN and Random Forest Confusion Matrices

Analysis of the models' confusion matrices (Figure 5.7) reveals a pattern similar to that observed with undersampling: although the classifiers perform nearly perfectly in distinguishing between the two non-"air" substances, they consistently fail to correctly classify ambient air. This limitation prevents the models from reliably detecting the presence of other substances.

5.3.4 Unbalanced, Re-Labelled

The classification inaccuracies between "air" and non-"air" substances can be partially attributed to imperfections in the experimental procedure, caused by two main factors:

1. The SmellInspector itself, where data is transmitted to the connected computer at inconsistent intervals. If the software continues labeling incoming data as a substance after it is no longer present, ambient air samples may be incorrectly labeled as the substance, and vice versa.

2. Human error during the experiment, as the user is responsible for updating the sample label and introducing the liquid onto the heating plate. Mistiming can result in air samples being labeled as substance or substance samples being labeled as "air". Additionally, since substance presence is inferred from air humidity, it is possible that the substance is no longer detectable in the surrounding air while samples are still labeled as substance. Samples labeled as substance before complete evaporation should also be re-labeled as air.

To mitigate these issues, the re-labeling algorithm described in Section 5.3 was applied. Out of the original 3489 "Domol" samples, 1999 were re-labeled as "air", and out of 2734 "Octeniderm" samples, 1526 were re-labeled.
Although the dataset for each substance was nearly halved, the remaining samples correspond to time points with the highest substance concentrations. After data normalization, lower concentration measurements are effectively filtered out, and the training data is balanced, preventing re-labeled outliers from negatively affecting air classification.
When evaluating the re-labeling algorithm on an imbalanced dataset (Table 5.7), classification scores for the substances decreased in favor of improved air classification scores (see Table 5.8). Given the reduced size of the substance datasets, this outcome was anticipated.

Group / Substance	Air	Domol	Octeniderm
Training	6593	1035	810
Test	3503	455	398

Table 5.7: Train / Test Sample Distributions

Despite the aggressive re-labeling, many air samples were still misclassified as substance, as evident in the models' confusion matrices (Figure 5.8). However, compared to the initial imbalanced classifiers (Section 5.3.1), the results exhibited slight overall improvement.

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	25.8%	36.1%	26.0%	36.8%
Octeniderm	36.0%	18.4%	47.7%	26.1%
Domol	23.4%	9.7%	25.4%	9.6%

Table 5.8: Statistics for re-labelled, unbalanced data

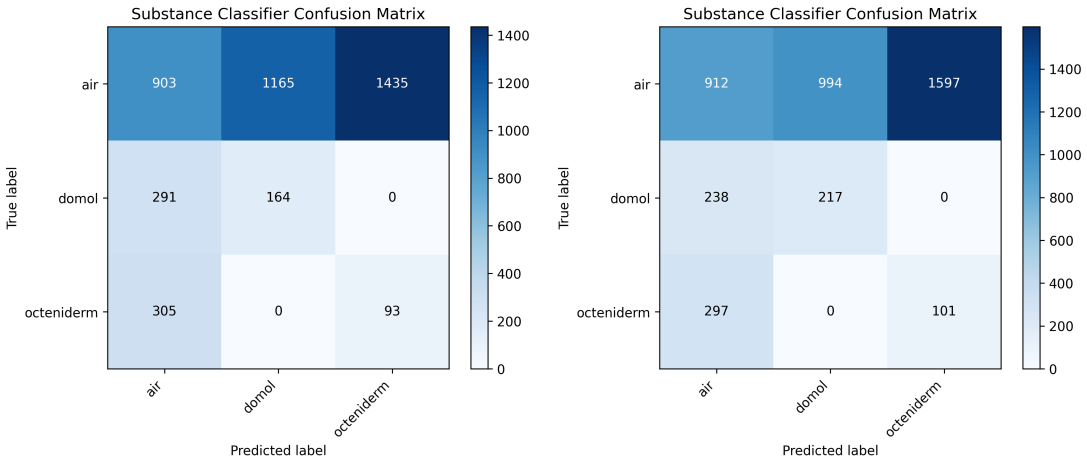


Figure 5.8: KNN and Random Forest Confusion Matrices

5.3.5 Oversampled, Re-Labelled

To address the imbalance and reduced substance counts resulting from the re-labeling algorithm, the classifier was re-configured to oversample the underrepresented substance classes, since oversampling previously resulted in the best metrics.
The resulting substance class distributions are shown in Table 5.9.

Group / Substance	Air	Domol	Octeniderm
Training	6593	6593	6593
Test	3503	3503	3503

Table 5.9: Train / Test Sample Distributions

When evaluated, classification performance improved moderately compared to the previous oversampling results. The F-score for "air" increased to 20.0% from the previous 12.4%, while "Octeniderm" improved to 53.9%, up from 44.5%. Classification of "Domol" decreased slightly to 33.6%, representing a drop of 4.1% from the previous 37.7%.

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	14.0%	12.2%	25.0%	20.2%
Octeniderm	54.0%	57.8%	46.9%	53.9%
Domol	30.2%	32.9%	29.9%	33.6%

Table 5.10: Statistics for oversampled, re-labelled data

Analysis of the corresponding confusion matrix (Figure 5.9) indicates that oversampling effectively reduced misclassification between non-"air" substances and slightly improved the detection of ambient air. However, misclassification between air and substance classes persists even after re-labeling and balancing the dataset.

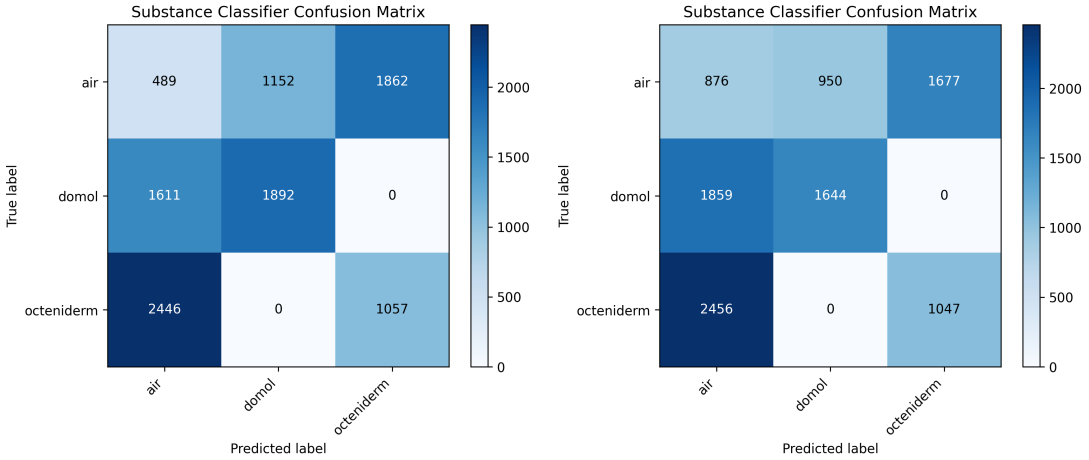


Figure 5.9: KNN and Random Forest Confusion Matrices

5.3.6 Undersampled, Re-Labelled

To mitigate potential biases introduced by oversampling, the evaluation was repeated using the undersampling strategy. Following re-labeling, each sample group was again reduced to 810 training samples and 398 test samples (see Table 5.11).

Group / Substance	Air	Domol	Octeniderm
Training	810	810	810
Test	398	398	398

Table 5.11: Train / Test Sample Distributions

Testing with this configuration produced the best classification results, with "Octeniderm" achieving an F-score of 67.4% and "Domol" reaching 52.3%. Classification of "air" samples remained challenging, yielding only 18.7%.

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	15.6%	16.4%	18.6%	18.7%
Octeniderm	61.8%	63.6%	65.1%	67.4%
Domol	60.3%	55.9%	54.3%	52.3%

Table 5.12: Statistics for undersampled, re-labelled data

Inspection of the confusion matrices (Figure 5.10) shows that the classifiers successfully distinguished between the two non-"air" substances, but continued to struggle with differentiating air from non-"air" samples. Nevertheless, compared to previous evaluations,

most non-“air” substance samples were now classified correctly. Despite these improvements, limitations in the data acquisition process (see Section 5.3.4) continued to adversely affect the accuracy of air classification.
Overall, these results demonstrate that the re-labeling algorithm was effective in reducing mislabeled substance samples and improving the reliability of non-“air” substance classification.

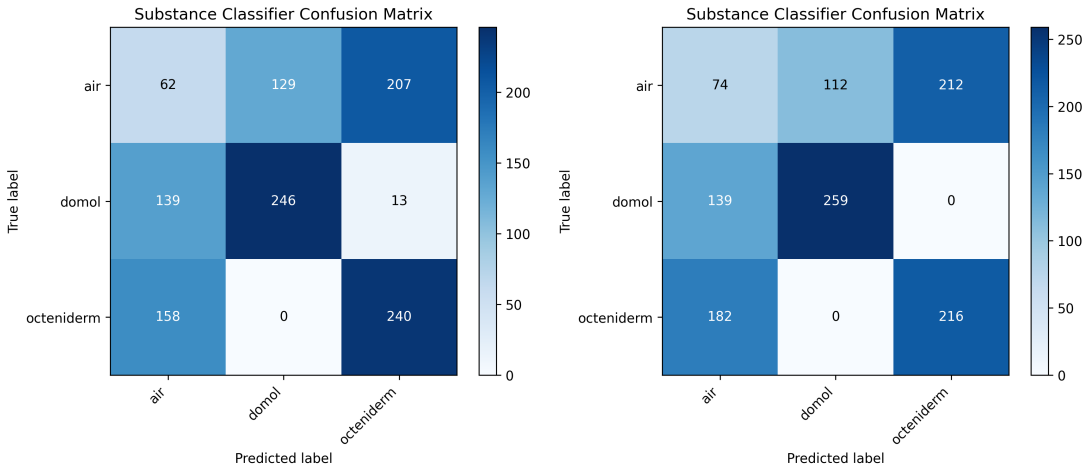


Figure 5.10: KNN and Random Forest Confusion Matrices

5.4 Substance Quantity Classification Results

While the initial procedure aimed first to achieve successful substance classification and then attempt quantitative classification, the latter produced lower than expected results. Consequently, quantitative analysis was performed using smaller datasets primarily as an exploratory experiment. Due to the limited quantity dataset size, evaluation of the quantitative classifiers employed the oversampling technique described previously, as undersampling led to artificially high scores due to model overfitting.
After oversampling, each substance class contained a total of 6593 samples, as shown in Table 5.13. The evaluation focused primarily on the “Octeniderm” substance, with “Domol” included as a non-“air” reference class to ensure that quantitative classification did not interfere with non-“air” substance classification.

Group / Substance	Air	Domol 50 μl	Octeniderm 50 μl	Octeniderm 100 μl
Training	6593	6593	6593	6593
Test	3503	3503	3503	3503

Table 5.13: Train / Test Sample Distributions

Metric evaluation revealed low classification scores for the two Octeniderm sample classes, with $50\mu l$ achieving an F-score of only 33.3%, while $100\mu l$ reached 26.3%, indicating that the classifier still struggled to distinguish between the presence of gaseous Octeniderm and ambient air. An overview of all classification scores is provided in Table 5.14.

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	14.2%	9.2%	25.6%	15.8%
Domol 50 μl	54.6%	57.5%	56.4%	61.2%
Octeniderm 50 μl	28.0%	32.8%	28.2%	33.3%
Octeniderm 100 μl	22.1%	34.0%	16.0%	26.3%

Table 5.14: Substance quantity classification statistics

When Octeniderm was evaluated against Domol, the classifier successfully distinguished between the two substances. Interestingly, the classifier also managed to differentiate between the two Octeniderm quantity classes, contrary to the manufacturer's claims (see the confusion matrices in Figure 5.11). This result may be influenced by biases introduced through re-labeling and aggressive oversampling.
Repeating the experiment with additional datasets produced similar classification metrics, suggesting that sample quantification may be achievable through extensive data manipulation.

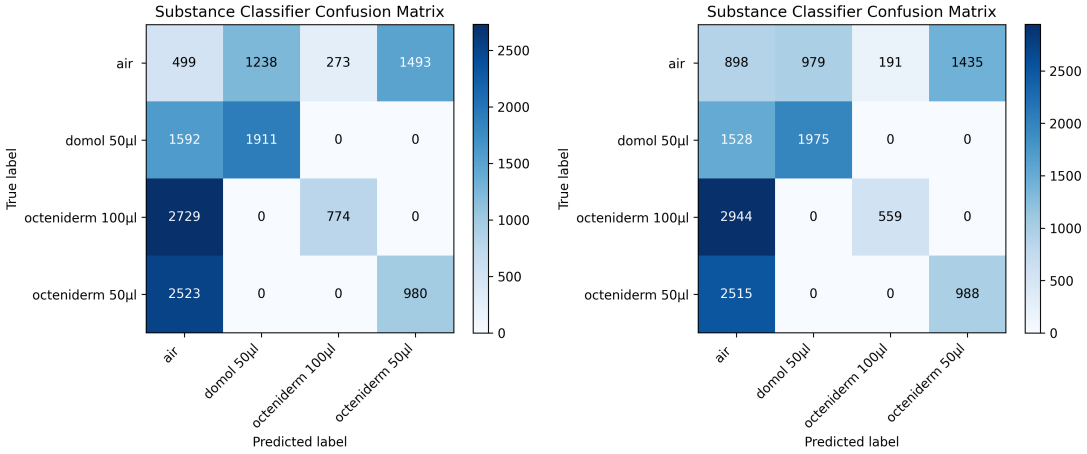


Figure 5.11: KNN and Random Forest Confusion Matrices

5.5 Humidity as a Feature?

In all previous experiments, humidity readings from the sensor were used solely to determine the presence of a substance for the purpose of data re-labeling, but were never included as input features for the classification models. This approach was intended to ensure that the classifiers learned to distinguish substances based solely on patterns detected by the nanotube sensor-arrays, without relying on humidity measurements from the integrated hygrometer, which naturally fluctuate depending on the environmental conditions of the sensor.
For clinical applications, one might argue that environmental conditions are highly regulated and controlled, leading to relatively stable humidity and temperature levels. In reality, however, in a NICU setting, although temperatures are controlled and air is filtered through air filtration systems, humidity often remains influenced by ambient conditions, including current weather and seasonal variations.

Nonetheless, an additional experiment was conducted to evaluate whether incorporating humidity as a feature could improve the classification results.
For this experiment, the re-labeling algorithm was applied to an undersampled, balanced dataset, as described in Section 5.3.6. Table 5.15 showcases the class distributions used for the experiment setting.

Group / Substance	Air	Domol	Octeniderm
Training	810	810	810
Test	398	398	398

Table 5.15: Train / Test Sample Distributions

In this experiment, incorporating humidity as a feature led to substantially higher classification scores compared to previous results, with "air" achieving 37.2%, "Octeniderm" 62.3%, and "Domol" 71.5%. This suggests a notable improvement in performance. When evaluated on other unseen datasets, the statistics remained similar, indicating that humidity may provide additional discriminatory information for substance classification. However, as noted previously, all datasets were collected within a relatively short timespan (2 Months), resulting in relatively consistent ambient humidity, which may have contributed to the observed stability in performance.

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	39.4%	39.4%	36.4%	37.2%
Octeniderm	74.1%	71.2%	62.8%	62.3%
Domol	62.8%	65.5%	72.4%	71.5%

Table 5.16: Classification Statistics when using Humidity as a Feature

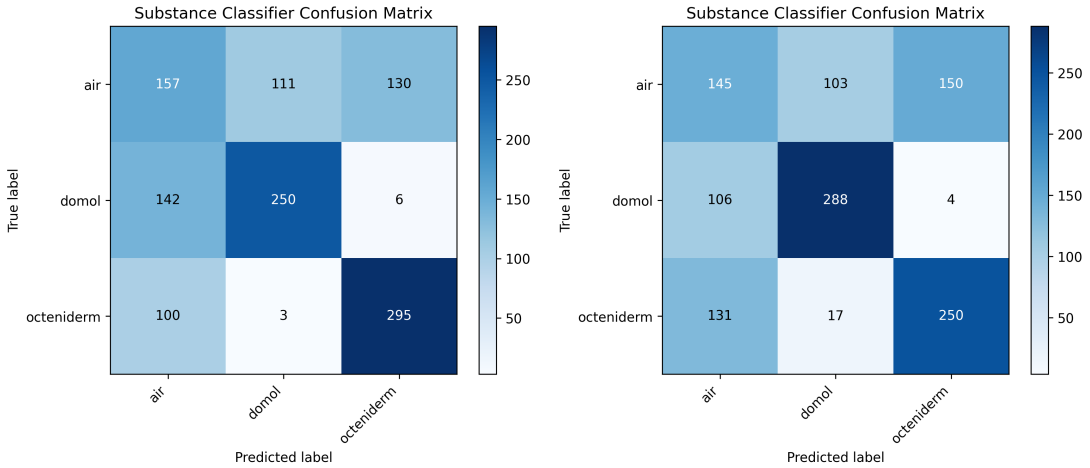


Figure 5.12: KNN and Random Forest Confusion Matrices

5.5.1 Humidity: Pitfalls

Although including humidity as a feature improved classification performance, it is not a reliable metric for in-situ deployment due to variable environmental conditions. Operating rooms may provide relatively stable conditions, but incubation rooms exhibit less control over ambient humidity. Furthermore, despite the observed statistical improvements, the practical use of humidity as a feature is limited by inaccuracies in the SmellInspector's humidity sensor and its independence from the smell.iX16 array's nanotube sensors, effectively allowing the hygrometer readings to overshadow the information provided by the nanotube sensors.
To assess sensor accuracy, an independent hygrometer was placed inside the measurement enclosure during experiments. On average, this external reference measured 5% to 10% lower humidity than the SmellInspector, with smaller fluctuations during liquid dispersion. Temperature readings also differed by up to $5°C$.
Consequently, humidity readings were deliberately excluded as input features for classification and were instead used solely as a filtering metric in the re-labeling algorithm, preventing the introduction of additional bias beyond that inherent in the measured data. Moreover, using humidity as a feature presents several practical challenges, ranging from temporal delays in substance dispersion, which make humidity a lagging and noisy indicator, to variations in ventilation, room size, and ambient temperature which can limit model generalizability across different settings. Furthermore, sensor drift causes reduced reliability over time, and small or oversampled datasets may lead to overfitting on humidity-specific patterns that do not reflect true substance signatures.
Collectively, these factors justify restricting the use of humidity to re-labeling rather than as a direct input for substance classification, especially since classification is intended to be performed using the nanotube arrays, not a hygrometer.

5.5.2 SmellAnnotator: Data Classification

Given the previous limitations and classification failures, the use of SmellInspector devices in a clinical routine had already been deemed impractical. To rule out potential errors arising from data handling or interpretation, another test was performed following the procedure used by the SmellAnnotator application.
While the custom software in earlier experiments trained ML models using each of the 64 sensor channels independently, the SmellAnnotator averages sample readings from four smell.iX16 sensors for each of the 16 channels before feeding them to the model during training. To evaluate the impact of this approach, the 64 channels were grouped into quartets, and the average values within each quartet were computed.
In Practice: Channels 1, 17, 33, and 49 are averaged to generate a single representative value, replacing the four individual readings. This averaging is based on the assumption that the four smell.iX16 chips operate similarly, even though, in practice, their readings often differ. The same procedure was applied to channels 2 to 16, 18 to 32, 34 to 48, and 50 to 64.
For this test, humidity was included as a feature to ensure full consistency with the SmellAnnotator methodology.

Table 5.17 presents the sample distribution across the different classes used for training and testing.

Group / Substance	Air	Domol	Octeniderm
Training	810	810	810
Test	398	398	398

Table 5.17: Train / Test Sample Distributions

When evaluated, both the KNN and Random Forest classifiers appeared to perform very well, with the RF classifer achieving F-scores of 67.1% for "air", 75.3% for "Domol", and 83.4% for "Octeniderm" (see Table 5.18). The corresponding confusion matrices 5.13 indicate that most samples were classified correctly, suggesting a superficially robust model.

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	68.6%	66.5%	70.1%	66.4%
Octeniderm	88.4%	84.8%	81.4%	79.2%
Domol	65.8%	71.1%	63.8%	69.6%

Table 5.18: Classification Statistics when Averaging across each smell.iX16 Sensor

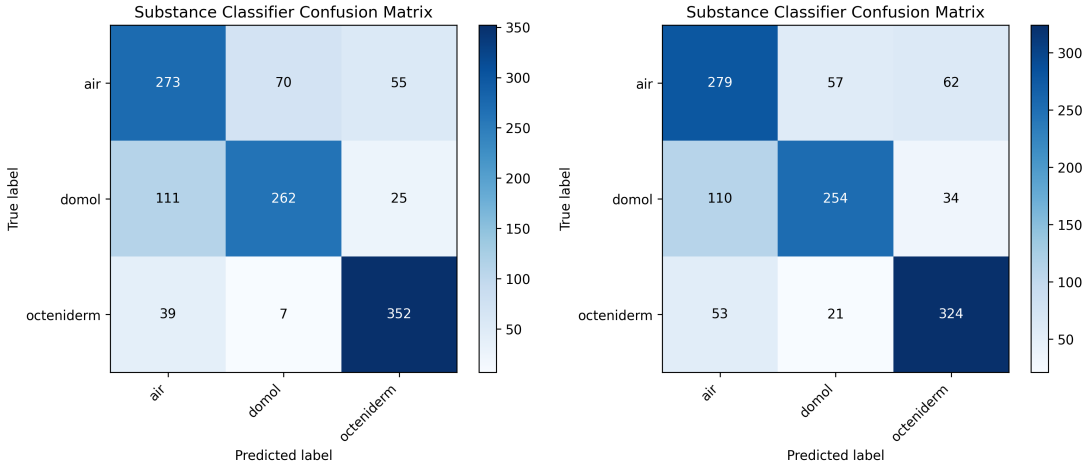


Figure 5.13: KNN and Random Forest Confusion Matrices

Closer inspection, however, reveals an underlying classification bias. Although the four smell.iX16 sensors are intended to function identically, raw measurements across channels differ substantially (Table 5.19), causing the averaged values to be dominated by one or two sensors in each quartet. As a result, the model effectively relies on a subset of sensors rather than all four.

Resistance Ω / smell.iX16 Sensor	CH1	CH2	CH3	CH4	CH5	CH6	CH7	CH8	CH9	CH10	CH11	CH12	CH13	CH14	CH15	CH16
Sensor 1	10489.3	10488.2	1353.4	1353.2	1353.3	4352.6	4353.4	4353.7	1751.4	1750.8	1750.9	4383.2	4383.6	4384.2	8714.3	8713.6
Sensor 2	7334.6	7335.0	3260.9	3260.5	3260.4	1072.9	1072.7	1073.0	1712.1	1711.2	1711.3	1799.9	1799.9	1800.3	5818.9	5818.7
Sensor 3	10382.8	10384.3	6459.0	6459.2	6459.5	1563.0	1562.6	1562.8	12406.3	12409.1	12408.4	8882.7	8883.7	8885.5	19334.7	19332.3
Sensor 4	4057.5	4055.5	1262.6	1261.9	1262.1	2124.2	2124.1	2124.4	1802.9	1802.5	1802.2	1512.0	1511.8	1512.1	8153.4	8153.1
Averaged	8066.0	8065.7	3083.9	3083.7	3083.8	2278.1	2278.2	2278.4	4418.1	4418.4	4418.2	4144.4	4144.7	4145.5	10505.3	10504.4

Table 5.19: Sensor Resistance Across All Four smell.iX16 Sensors in Contrast to Computed Average Resistance

Moreover, removing humidity as a feature causes a dramatic drop in performance (Table 5.20), indicating that the model depends heavily on the integrated hygrometer readings rather than the nanotube sensor measurements themselves.
As observed in previous experiments, this suggests that the apparent high performance is largely driven by environmental humidity rather than intrinsic sensor response.

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	12.1%	12.5%	13.1%	11.1%
Octeniderm	61.1%	56.2%	31.2%	33.2%
Domol	58.8%	61.7%	47.0%	53.0%

Table 5.20: Classification Statistics when Averaging across each smell.iX16 Sensor **without** using Humidity

5.5.3 Classification Based On Humidity Alone

With the drastic improvement in model classification after introducing humidity as a feature, it begs the question: Can the model classify substances based on humidity alone, effectively rendering the nanotube measurements obsolete?
To test this, the previous test was repeated using 1-dimensional data vectors with each sample represented using its humidity value alone. For the train/test split, the dataset from the previous test 5.17 was used again (undersampled, re-labeled).

Substance	KNN		Random Forest	
	Recall	F-score	Recall	F-score
Air	75.1%	73.4%	74.4%	70.9%
Octeniderm	54.3%	55.5%	54.3%	54.4%
Domol	50.8%	44.2%	63.8%	46.4%

Table 5.21: Classification Statistics when using only Humidity

When tested, both substance classifications performed worse than before, but achieved impressive classifcation statistics in contrast to earlier tests, with “Octeniderm” achieving an F-score of 54.4% and “Domol” achieving a score of 46.4%. For “air” classifcation, the models yielded the second best result overall, with an F-score of 70.9%.
Comparing the models’ confusion matrices 5.14 reveals impressive air to substance classifcation accuracy, with cross-sample classification lowering the overall model’s performance.

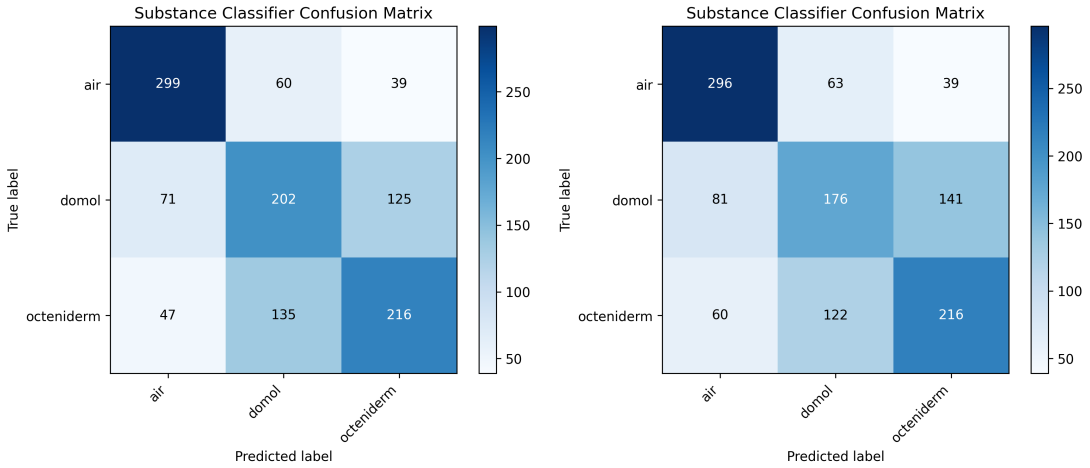


Figure 5.14: KNN and Random Forest Confusion Matrices

These results confirm the previously assumed model bias toward relying primarily on humidity as the dominant feature, while using the nanotube sensor values mainly to differentiate between substance classes. Although this behavior may yield stable performance in environmentally controlled settings, it proves unreliable when the smell.iX16 sensors are operated independently due to the absence of an integrated hygrometer. Consequently, the models fail to generalize under fluctuating environmental conditions, such as varying temperature and humidity levels.

5.6 Discussion

Although the classifiers were able to differentiate between substance classes and quantities, they consistently failed to classify ambient air correctly, rendering them impractical for real-world use. Introducing humidity as a feature improved results, since samples containing diluted substances exhibited higher humidity readings. However, this approach is only viable in strictly controlled environments and is unreliable in in-situ settings, where ambient humidity fluctuates with season and weather conditions. Furthermore, the models' inherent dependence on humidity as a key feature effectively precludes the deployment of smell.iX16 sensors as standalone sensors, since they lack an integrated humidity sensor necessary for accurate classification.

5.6.1 Varying Sensor Reactivity

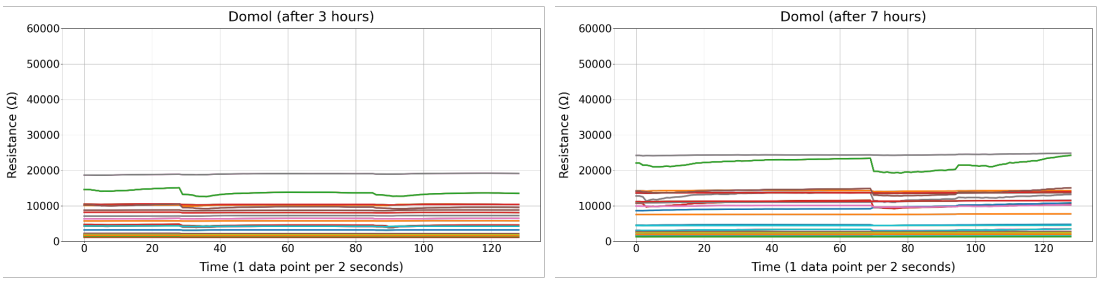


Figure 5.15: Sensor Reactivity to "Domol" After 3 Hours (Left) vs. After 7 Hours (Right)

Additionally, sensor reactivity to the same substances varied over prolonged exposure, with resistance values drifting over several hours and environmental conditions further influencing measurements. Over time, resistance across all 64 channels increased by several thousand ohms, and fluctuations in response times to sample introduction became more erratic, demonstrating inherent sensor instability, as shown in Figure 5.15.

5.6.2 Sensor Aging

Beyond continuous drift and variable reactivity, defective sensor performance observed during initial experiments 3.4 raises concerns about the longevity and reliability of the device under continuous use. Without calibration, the older sensor continued producing erratic data and did not recover to baseline performance. With no apparent cause for these defects and manufacturer assurances that the device was functioning nominally, the SmellInspector cannot be relied upon for rigorous in-situ deployment.

5.6.3 Larger Test Medium

Scaling the test environment revealed additional limitations. When the SmellInspector was moved from the small test container to a larger volume [1] simulating a real incubator, the sensor failed to register any detectable response at previously effective substance concentrations. While mounting the sensor to the Incubator's exhaust fan might mitigate this, the SmellInspector's vertical mounting requirement and bulkiness makes such positioning infeasible, restricting placement to incubator walls.

5.6.4 Conclusion

Overall, the SmellInspector demonstrated the potential to classify certain substances under controlled experimental conditions. However, several factors limit its practical use in real-world, in-situ applications. Environmental fluctuations, sensor aging and drift, variability across the four integrated smell.iX16 sensors, and reduced sensitivity in larger test volumes all contribute to inconsistencies in classification. Moreover, the reliance on humidity as a feature suggests that observed performance gains may be driven more by environmental cues than by the intrinsic response of the nanotube sensors themselves.
Despite these limitations, the SmellInspector shows promise for carefully controlled research settings, particularly where extensive datasets and stable environmental conditions can be maintained. For broader or clinical deployment, however, alternative gas detection technologies, such as gas spectroscopes or iron-oxide sensors currently provide more robust and reliable performance. In this context, the SmellInspector can be considered a valuable experimental tool for exploring nanosensor-based odor and VOC detection on a broader range, while further development would be needed to achieve practical applicability in dynamic real-world environments.

[1] An IKEA SAMLA Box [33], measuring 57×39×28cm / 45l

Chapter 6

Conclusion and Future Plans

The SmellInspector demonstrated only limited capability in detecting and classifying disinfectant vapors dispersed in ambient air, restricting its practical use to tightly controlled laboratory environments. Under such conditions, the device achieved promising classification performance, suggesting that nanosensor-based olfactory monitoring holds conceptual potential, while requiring significant refinement before broader deployment is feasible.

The evaluated machine learning models (RF and KNN) revealed a strong dependence on ambient humidity as a discriminative feature. Though incorporating humidity improved overall classification accuracy, it introduced an uncontrollable bias, as humidity naturally fluctuates with temperature, ventilation, and weather conditions. Consequently, the improved accuracy came at the expense of generalizability. Without humidity as an input, model performance dropped sharply, confirming that classification success stemmed primarily from environmental correlations rather than intrinsic nanotube sensor sensitivity.

Beyond environmental dependencies, technical challenges were also observed. Sensor drift, cross-channel resistance variability, and inconsistent baseline values occurred even within the same device, while uncontrolled sensor aging rendered devices unusable after some time. These factors make long-term operation impractical. Nevertheless, the sensors consistently exhibited distinguishable response patterns under stable conditions, supporting their continued exploration for controlled experimental use.

Data preprocessing methods provided incremental improvements in robustness but could not mitigate fundamental sensor reliance on ambient conditions. The re-labelling algorithm effectively corrected misclassifications caused by user errors during data acuisition, yet the models continued to struggle to separate air samples from substance samples. Although the classifiers could differentiate between distinct substances using nanotube data alone, their heavy reliance on humidity for substance detection revealed the sensors to provide insuffient data if operated independently.

When the test medium was scaled to mimic real incubators, the sensor consistently failed to register the presence of tested substances, further limiting its employability.

In summary, although the SmellInspector is not yet viable for clinical deployment, it serves as a valuable research platform and proof of concept for olfactory research. The study underscores the importance of distinguishing between model-driven statistical accuracy and physically meaningful sensor performance. Though machine learning can enhance data interpretability it cannot compensate for underlying hardware constraints. Future research will therefore focus on more stable and sensitive sensor technologies using metal-oxide gas sensors.

The SmellInspector may continue to serve as a flexible testbed under controlled settings. Ultimately, aside from evaluating the sensor's capability under fluctuating environmental conditions, none of this work's goals could be considered fulfilled. Substance classification worked in a small test medium only in highly controlled environments, effectively ruling out the feasibility of real-time monitoring systems and clinical integration. Still the work highlights both the promises and limitations of nanosensor-based olfactory sensory, and the need for further research. Although the current implementation falls short of clinical applicability, it represents a crucial step toward integrating functional olfactory sensing into the clinical routine. With continued refinement of both sensor hardware and data processing, the long-term vision of continuous olfactory monitoring in neonatal care remains an achievable goal.

Chapter 7

Appendix

This appendix acts as installation reference for deploying the SmellInspector-Companion software in other research works. It gives a walkthrough of the installation process and showcases adjustable configurations for training machine learning models.

7.1 Installation and Setup

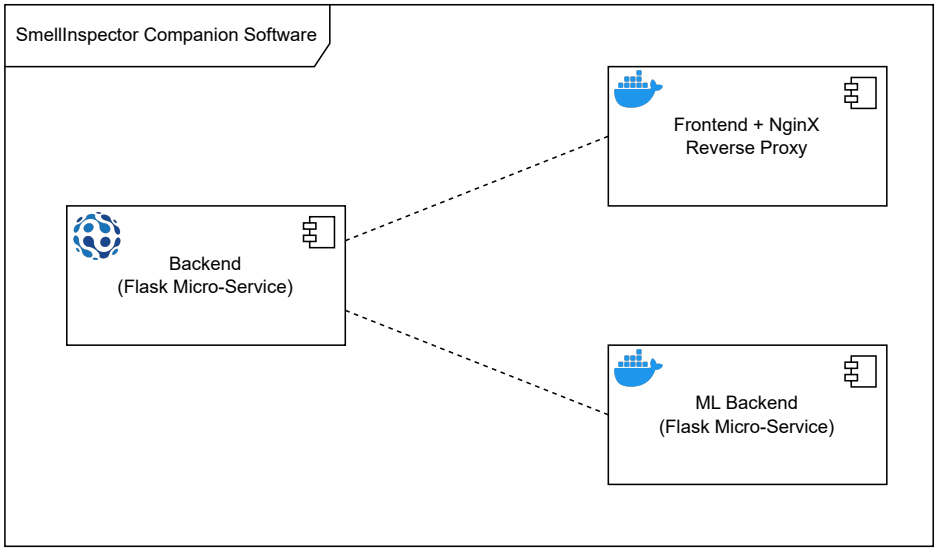


Figure 7.1: SmellInspector-Companion Software Abstract Architecture

As described in Chapter 4, the SmellInspector-Companion is comprised of three independant software components, the "Backend", "Frontend" and "Machine Learning" components (as illustrated in figure 7.1).
To run the software, clone or download the repository [32] on the machine that the SmellInspector devices will be connected to (physically, a VM will not work!).

7.1.1 Required Software and Drivers

Ensure that the following software is installed on the host machine:

- Python 3 version 3.11 or later [34]. Earlier versions **will fail to compile** due to unsupported type hints and imports.
- Docker [35] with the compose plugin [36] enabled.

7.1.2 UART Driver Installation

Before configuring the software, install the CP210x USB to UART Bridge VCP Drivers from Silicon Labs [37]. These are required to receive incoming UART data from connected SmellInspector devices. If the driver is not installed, the port scan on the "Add Device" page 4.3.4 will fail and prompt for a driver installation message.

7.1.3 Web-Configuration

By default, the software is started in localhost mode. For normal test scenarios, this makes the most sense, since SmellInspector devices need to be manually operated and remote management makes little sense.
If the device is however deployed in a live environment, the software can be controlled over a local network as well. To enable this, locate the file *frontend/src/authentication/global-vars.ts* and change the *defaultIP* variable to your local IPv4 address.
Connectivity over foreign networks is also enabled through the use of an NginX Reverse Proxy. The default configuration file is located at *frontend/nginx.conf* and enabled by default.
The default port for serving the web-application is 80/HTTP. To change this, edit the NginX configuration accordingly.

7.1.4 Machine Learning Configurations

The machine learning component also offers various parameters as illustrated in Chapter 5. These can all be configured through the provided *.env.example* file in the *machine_ learning* directory. To enable the custom configurations, copy the provided example file to *machine_ learning/.env* and then adjust the included parameters as desired. The configurations are loaded automatically when the application is started.
The adjustable parameters are:

- Re-Training Rate *"RE_ TRAINING_ RATE"*: How many samples are received from the "backend" component before triggering a model re-training (if enabled for the current test).
- Balance Datasets *"BALANCE"*: Whether datasets should be balanced (True) or imbalanced (False). If left empty, datasets are balanced by default.
- Balance Strategy *"BALANCE_ STRATEGY"*: What strategy should be used to balance the datasets. Available values are "OVERSAMPLE", "UNDERSAMPLE" or empty for default (Oversample).

- Humidity-as-a-Feature *"ENABLE_ HUMIDITY"*: Whether humidity should be included as a feature, see Chapter 5. Enabled by default (True).
- Use Averages *"USE_ AVERAGES"*: Whether the data gathered from the 4 smell.iX16 sensors should be averaged to produce a single feature over four raw ones. Enabled by default (True).

In the default configuration state, the models are trained analogously to the machine learning models in SmartNanotube's SmellAnnotator software, using humidity as a feature and averaging sensor values rather than using raw data. This was done to ensure consistency across applications. The configurations are provided to allow experimentation and research without guardrails as those present in the SmellAnnotator software.

7.1.5 First Start

After applying the web- and machine-learning configurations, the software is ready for deployment.
The startup sequence for the software components should always be executed in the following order to ensure data consistency across the backend and machine learning components:

1. The frontend is started first, launching an NginX Reverse Proxy for proxying data across the three components.
2. The machine learning backend, ensuring a clean startup with no cached data.
3. The backend is started lastly. On start, existing database data is migrated to the machine learning component.

To start the application, navigate to the root of the repository and execute the command:

```
docker compose up --build -d
```

This starts the frontend and machine learning components.
To start the backend, navigate to the *"backend"* subdirectory and create a new Python virtual environment. Then activate it, install the required libraries and execute the software:

```
cd backend/
python3 -m venv .venv
# Windows
./.venv/Scripts/activate.ps1
# MacOs / Linux
source ./.venv/bin/activate
pip3 install -r requirements.txt
python3 app.py
```

The backend will start, synchronise data with the machine learning backend, prompting an initial model training.
After startup, the application will be available at:

```
http://localhost/ - for the host's machine.
http://your_IPv4_address/ - for clients on the same network.
```

Disclaimer: The application is not intended for public hosting. Proceed with caution.

7.2 Classification Results Overview

As mentioned in Chapter 5, the following table gives an overview over all used configurations and their corresponding classification statistics.

Humidity Setting	Label Configuration	Balance Strategy	Substance	Model			
				KNN		Random Forest	
				Recall	F-Score	Recall	F-Score
No Humidity	No Re-Labeling	Unbalanced	Air	19.6%	19.5%	13.7%	14.7%
			Octeniderm	33.3%	29.5%	43.3%	36.0%
			Domol	22.2%	27.4%	24.5%	28.7%
		Undersampled	Air	11.6%	9.4%	10.0%	8.3%
			Octeniderm	33.9%	34.1%	39.7%	38.7%
			Domol	29.9%	39.0%	30.0%	38.8%
		Oversampled	Air	16.5%	13.1%	14.9%	12.4%
			Octeniderm	37.4%	37.4%	46.8%	44.5%
			Domol	26.5%	36.0%	28.3%	37.7%
	Re-Labeled	Unbalanced	Air	25.8%	36.1%	26.0%	36.8%
			Octeniderm	36.0%	18.4%	47.7%	26.1%
			Domol	23.4%	9.7%	25.4%	9.6%
		Undersampled	Air	15.6%	16.4%	18.6%	18.7%
			Octeniderm	61.8%	63.6%	65.1%	67.4%
			Domol	60.3%	55.9%	54.3%	52.3%
		Oversampled	Air	14.0%	12.2%	25.0%	20.2%
			Octeniderm	54.0%	57.8%	46.9%	53.9%
			Domol	30.2%	32.9%	29.9%	33.6%
Humidity Enabled	Re-Labeled	Undersampled	Air	39.4%	39.4%	36.4%	37.2%
			Octeniderm	74.1%	71.2%	62.8%	62.3%
			Domol	62.8%	65.5%	72.4%	71.5%
	Re-Labeled	Undersampled, Averaged	Air	68.6%	66.5%	70.1%	66.4%
			Octeniderm	88.4%	84.8%	81.4%	79.2%
			Domol	65.8%	71.1%	63.8%	69.6%
No Humidity	Re-Labeled	Undersampled, Averaged	Air	12.1%	12.5%	13.1%	11.1%
			Octeniderm	61.1%	56.2%	31.2%	33.2%
			Domol	58.8%	61.7%	47.0%	53.0%
Humidity only	Re-Labeled	Undersampled	Air	75.1%	73.4%	74.4%	70.9%
			Octeniderm	54.3%	55.5%	54.3%	54.5%
			Domol	50.8%	44.2%	63.8%	46.4%

Table 7.1: Classification Results Overview

Bibliography

[1] "SmartNanotubes Technologies GmbH new Homepage (2025)," last accessed: 16. August, 2025. [Online]. Available: https://www.smart-nanotubes.com/#specs

[2] "SmellAnnotator for Windows | SmartNanotubes GmbH," last accessed: 10. October, 2025. [Online]. Available: https://s3.eu-central-1.amazonaws.com/downloads.smartnanotubes.com/Smell_Annotator_Setup_1.5.0.exe

[3] "SmellAnnotator Android Application (Desktop inaccessible) | Google Play Store," last accessed: 10. October, 2025. [Online]. Available: https://play.google.com/store/apps/details?id=com.smartnanotubes.smellannotator&hl=en&pli=1

[4] Hinz, C. and Heep, A. and Plösch, T. and Huber, J. and Lange, M., "Review identifies odorants used in neonatal clinical care but identifies need for further research," *Acta Paediatrica*, vol. 113, no. 10, pp. 2216–2219, 2024, Epub 2024 Jun 28. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/38940201

[5] Roberts, S. Craig and Havlíček, Jan and Schaal, Benoist, "Human olfactory communication: current challenges and future prospects," *Philosophical Transactions of the Royal Society B: Biological Sciences*, vol. 375, no. 1800, p. 20190258, 2020, epub 2020 Apr 20. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/32306869

[6] Jessen, S., "Maternal odor reduces the neural response to fearful faces in human infants," *Dev Cogn Neurosci*, vol. 45, p. 100858, 2020, Epub 2020 Sep 8. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/32927245

[7] Sarnat, H. B. and Flores-Sarnat, L., "Development of the human olfactory system," in *Handb Clin Neurol*, 2019, vol. 164, pp. 29–45. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/31604554

[8] Schaal, B., Marlier, L., and Soussignan, R., "Human foetuses learn odours from their pregnant mother's diet," *Chem Senses*, vol. 25, no. 6, pp. 729–737, 2000. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/11114151

[9] Cheong, J. L. Y. and Burnett, A. C. and Treyvaud, K. and Spittle, A. J., "Early environment and long-term outcomes of preterm infants," *Journal of Neural Transmission (Vienna)*, vol. 127, no. 1, pp. 1–8, 2020, Epub 2019 Dec 20. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/31863172

[10] Berkhout, D. J. C. and Niemarkt, H. J. and Buijck, M. and van Weissenbruch, M. M. and Brinkman, P. and Benninga, M. A. and van Kaam, A. H. and Kramer, B. W. and Andriessen, P. and de Boer, N. K. H. and de Meij, T. G. J., "Detection of Sepsis in Preterm Infants by Fecal Volatile Organic Compounds Analysis: A Proof

of Principle Study," *J Pediatr Gastroenterol Nutr*, vol. 65, no. 3, pp. e47–e52, Sep 2017. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/27846067

[11] de Kroon, R. R., Frerichs, N. M., Struys, E. A., de Boer, N. K., de Meij, T. G. J., and Niemarkt, H. J., "The Potential of Fecal Volatile Organic Compound Analysis for the Early Diagnosis of Late-Onset Sepsis in Preterm Infants: A Narrative Review," *Sensors (Basel)*, vol. 24, no. 10, p. 3162, May 16 2024. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/38794014

[12] Berkhout, D. J. C., Niemarkt, H. J., de Boer, N. K. H., Benninga, M. A., and de Meij, T. G. J., "The potential of gut microbiota and fecal volatile organic compounds analysis as early diagnostic biomarker for necrotizing enterocolitis and sepsis in preterm infants," *Expert Rev Gastroenterol Hepatol*, vol. 12, no. 5, pp. 457–470, May 2018, Epub 2018 Mar 6. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/29488419

[13] Steinbach, J., Goedicke-Fritz, S., Tutdibi, E., Stutz, R., Kaiser, E., Meyer, S., Baumbach, J. I., and Zemlin, M., "Bedside Measurement of Volatile Organic Compounds in the Atmosphere of Neonatal Incubators Using Ion Mobility Spectrometry," *Front Pediatr*, vol. 7, p. 248, Jun 18 2019. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/31275910

[14] "Picture of an Incubator," last accessed: 10. October, 2025. [Online]. Available: https://www.insuedthueringen.de/inhalt.fruehchen-schnelle-desinfektion-fuer-den-fruehchen-inkubator.8f30c34e-83b8-47fd-a915-8bcb567882dc.html

[15] "Panes-Ruiz, L.A., Riemenschneider, L., Al Chawa, M.M. et al. Selective and self-validating breath-level detection of hydrogen sulfide in humid air by gold nanoparticle-functionalized nanotube arrays. Nano Res. 15, 2512-2521 (2022). ," last accessed: 16. August, 2025. [Online]. Available: https://doi.org/10.1007/s12274-021-3771-7

[16] "SmellInspector Preview by SmartNanotubes | CES 2021 (Archive)," last accessed: 10. October, 2025. [Online]. Available: https://chiron-k9.com/wp-content/uploads/2021/03/SmartNanotubes_CES2021.pdf

[17] "SmellInspector Technical Manual by SmartNanotubes, version 1.3.0," last accessed: 26. November, 2024. [Online]. Available: https://smart-nanotubes.com/wp-content/uploads/2023/03/SNT_Manual_Smell-Inspector_V1.3.0.pdf

[18] "Arbeitsgruppe Perinatale Neurobiologie // Universität Oldenburg," last accessed: 19. August, 2025. [Online]. Available: https://uol.de/kinder-jugendmedizin/arbeitsgruppe-perinatale-neurobiologie

[19] "G. Wiederhold, J. McCarthy. Arthur Samuel: Pioneer in machine learning. IBM Journal of Research and Development, vol. 36, no. 3, pp. 329-331, 1992. DOI: 10.1147/rd.363.0329."

[20] "I. Jolliffe. Principal component analysis. International Encyclopedia of Statistical Science, M. Lovric, Ed., Berlin, Germany: Springer, 2011. DOI: 10.1007/978-3-642-04898-2455."

[21] "G. J. McLachlan. Discriminant Analysis and Statistical Pattern Recognition, New York: USA, Wiley, 2005."

[22] Evelyn Fix and J.,L.,Hodges Jr., "Discriminatory analysis: Nonparametric discrimination - Consistency properties," USAF School of Aviation Medicine, Randolph Field, Texas, Tech. Rep. Report No. 4, Project No. 21-49-004, Feb. 1951.

[23] "KNN visualization in just 13 lines of code | by Deepthi A R | TDS Archive | Medium," last accessed: 10. October, 2025. [Online]. Available: https://medium.com/data-science/knn-visualization-in-just-13-lines-of-code-32820d72c6b6

[24] Breiman, Leo, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. [Online]. Available: https://link.springer.com/article/10.1023/A:1010933404324

[25] Murthy, Sreerama K. and Kasif, Simon and Salzberg, Steven, "A system for induction of oblique decision trees," *J. Artif. Int. Res.*, vol. 2, no. 1, pp. 1–32, Aug. 1994.

[26] Tin Kam Ho, "Random decision forests," in *Proceedings of 3rd International Conference on Document Analysis and Recognition*, vol. 1, 1995, pp. 278–282 vol.1.

[27] "Conceptual visualization of a random forest. | ResearchGate," last accessed: 10. October, 2025. [Online]. Available: https://www.researchgate.net/figure/Conceptual-visualization-of-a-random-forest-It-is-made-up-of-a-several-decision-trees_fig4_345813508

[28] "SmellInspector Holder by Leonard Haddad | Makerworld," last accessed: 31. August, 2025. [Online]. Available: https://makerworld.com/en/models/1750746-smartnanotubes-smellinspector-sensor-holder#profileId-1861257

[29] "octeniderm® farblos - schülke," last accessed: 10. October, 2025. [Online]. Available: https://www.schuelke.com/de-de/produkte/912844/octeniderm-farblos

[30] "domol hygiene-spray | rossmann.de," last accessed: 10. October, 2025. [Online]. Available: https://www.rossmann.de/de/gesundheit-domol-hygiene-spray/p/4305615088181

[31] "Over-the-shelf Coffee Heating Plate from Amazon," last accessed: 10. October, 2025. [Online]. Available: https://www.amazon.de/dp/B08PYFLYJX

[32] "SmellInspector Companion Software by Leonard Haddad | GitHub," last accessed: 16. August, 2025. [Online]. Available: https://github.com/leolion3/SmartNanoTubes-SmellInspector-Comapnion

[33] "SAMLA Box with Lid | IKEA Germany," last accessed: 10. October, 2025. [Online]. Available: https://www.ikea.com/de/de/p/samla-box-mit-deckel-transparent-s69440761

[34] "Python.org Homepage," last accessed: 10. October, 2025. [Online]. Available: https://www.python.org

[35] "Docker Homepage," last accessed: 10. October, 2025. [Online]. Available: https://www.docker.com

[36] “Docker Compose | Docker Docs,” last accessed: 10. October, 2025. [Online]. Available: https://docs.docker.com/compose

[37] “CP210x USB to UART Bridge VCP Drivers - Silicon Labs,” last accessed: 10. October, 2025. [Online]. Available: https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers